DAM/DAW — ENTORNOS DE DESARROLLO — 1ER CURSO

Desarrollo de Software

Cómo una idea se convierte en un programa que funciona: del código fuente al proceso en memoria, con las fases, los lenguajes, las herramientas y las metodologías que intervienen.

RA1 — Resultado de Aprendizaje 1

CE1.a Programas y sistema informático | CE1.b Fases del desarrollo | CE1.c Fuente, objeto y ejecutable | CE1.d Código intermedio y máquinas virtuales

Saltar al contenido principal

CE1.e Clasificación de lenguajes | CE1.f Herramientas de desarrollo | CE1.g Metodologías ágiles

1.1

Programas y sistema informático

Qué ocurre en el ordenador cuando se ejecuta un programa: procesador, memoria, periféricos y sistema operativo.

CE1.a — Programas y componentes del sistema informático

> FUNDAMENTO — PROGRAMA Y PROCESO
>
> Un **programa** es un fichero en disco con instrucciones y datos (en Linux, formato ELF). Un **proceso** es ese programa cargado en memoria y en ejecución: tiene su espacio de direcciones, sus registros y sus recursos abiertos. El programa nunca maneja el hardware directamente: pide servicios al **núcleo** del sistema operativo. Este bloque sigue el camino de una instrucción desde el código hasta los componentes físicos.

Vocabulario para hablar con precisión

Software es el conjunto de programas, datos, configuración y documentación que permite a un sistema informático realizar funciones. Un **programa** es una secuencia organizada de instrucciones; una **aplicación** ofrece funciones a personas u otros sistemas y suele integrar varios programas. Estos términos no son sinónimos, y en el módulo conviene usarlos con exactitud porque cada criterio de evaluación se apoya en ellos.

TÉRMINO	IDEA PRINCIPAL	EJEMPLO
Script	Automatización pequeña y acotada, normalmente interpretada	Un fichero que renombra las fotos de una carpeta
Programa	Secuencia de instrucciones que una máquina puede ejecutar, directa o indirectamente, con un propósito	Un conversor de unidades
Aplicación	Producto que ofrece funciones a personas u otros sistemas; suele integrar varios programas, datos y servicios	El gestor académico del centro
Proyecto	Esfuerzo temporal con alcance, recursos, plazo y restricciones	Construir la versión 1 de un gestor de tareas de clase
Producto	Solución que vive más allá del proyecto y evoluciona mediante versiones	Ese gestor de tareas durante varios cursos
Sistema	Conjunto de software, hardware, datos, personas y procesos que cooperan	La plataforma educativa completa, con sus servidores y su personal

Una distinción más profunda separa **programar**, **desarrollar software** e **ingeniería del software**. Programar es diseñar algoritmos y escribir código. Desarrollar software incluye además entender la necesidad, planificar, diseñar, comprobar, desplegar, documentar, operar y mejorar. La ingeniería del software aplica principios sistemáticos y medibles para controlar la complejidad, el riesgo, el coste y la calidad de todo ese recorrido.

La cantidad de proceso debe ser proporcional al riesgo. Un script personal no necesita la trazabilidad de un sistema sanitario, pero ambos ganan mucho si se entiende el problema, se versionan los cambios y se comprueba el resultado.

Tipos de software y qué papel juega cada uno

Para relacionar un programa con los componentes del sistema informático hay que saber primero en qué categoría está, porque de ella depende con qué habla. Una aplicación se apoya en el sistema operativo, este en el firmware y los controladores, y todo ello en el hardware; el software de programación es el que se usa para producir el resto.

CATEGORÍA	FUNCIÓN	EJEMPLOS
Software de sistema	Administra los recursos del equipo y ofrece servicios básicos a los demás programas	Sistema operativo, controladores, firmware
Software de aplicación	Resuelve una necesidad de la persona usuaria o de la organización	Navegador, hoja de cálculo, ERP, videojuego
Software de	Ayuda a crear, comprobar y mantener otro software	IDE, compilador, depurador, Git

Saltar al contenido principal

CATEGORÍA	FUNCIÓN	EJEMPLOS
Middleware	Conecta aplicaciones, servicios o datos que no se conocen entre sí	Servidor de aplicaciones, gestor de colas de mensajes

Según su adaptación, un producto puede ser **estándar** (una misma solución para muchas organizaciones), **configurable** (se adapta con parámetros) o **a medida** (se construye para un cliente concreto). Esa clasificación no determina la licencia: software libre, código abierto, gratuito, propietario y servicio en la nube (SaaS) describen dimensiones distintas y deben analizarse por separado.

Los componentes físicos, desde el punto de vista del software

El hardware importa a quien programa por lo que condiciona. Los componentes que intervienen al ejecutar una aplicación son los siguientes.

COMPONENTE	QUÉ HACE	POR QUÉ IMPORTA A QUIEN PROGRAMA
CPU	Ejecuta instrucciones máquina y coordina las operaciones	Fija la arquitectura (x86-64, ARM) para la que se compila un ejecutable nativo
Memoria RAM	Guarda temporalmente el código y los datos de los procesos en uso; es rápida y volátil	Todo lo que no se guarda en almacenamiento se pierde al cerrar el programa
Almacenamiento	Conserva ejecutables, configuración y datos de forma persistente	Es donde vive el fichero del programa antes de ser proceso, y donde se guardan los datos que deben sobrevivir
Periféricos	Permiten la entrada, la salida, la comunicación y el almacenamiento externo	Nunca se manejan directamente: se accede a ellos mediante el núcleo y sus controladores
GPU y aceleradores	Realizan en paralelo cargas gráficas, científicas o de inteligencia artificial	Cambian la forma de programar ciertas tareas; requieren bibliotecas y controladores específicos
Red	Comunica procesos y máquinas mediante interfaces y protocolos	Convierte una aplicación en un sistema distribuido, con latencias y fallos que no existen en local

Entre la CPU y la RAM existe una **jerarquía de memoria**: registros, cachés y memoria principal, cada nivel más grande y más lento que el anterior. El almacenamiento persistente está todavía más lejos. Un buen modelo mental es que el programa nunca opera «sobre el disco»: el sistema operativo trae datos a la RAM, la CPU los procesa y después se vuelven a guardar.

APP	Nivel 1 **Tu programa (el proceso)**	Instrucciones y datos del programa, con su propio espacio de direcciones.	ELF en disco
LIB	Nivel 2 **Bibliotecas y entorno de ejecución**	Código reutilizable que el programa enlaza o carga: libc, la JVM, el intérprete de Python.	libc, JVM
SO	Nivel 3 **Núcleo del sistema operativo**	Único código autorizado a tocar el hardware: memoria virtual, procesos, ficheros y controladores.	Linux
HW	Nivel 4 **Hardware: CPU, memoria y periféricos**	Procesador, memoria RAM y dispositivos conectados por buses.	x86-64, ARM

Regiones del espacio de direcciones de un proceso

REGIÓN	CONTENIDO	PERMISOS	EN MEMORIA.C
.text	Instrucciones máquina del programa	`r-x` (leer y ejecutar)	el código de `suma` y `main`
.rodata	Constantes y cadenas literales	`r--` (solo lectura)	las cadenas del `printf`
.data	Variables globales inicializadas	`rw-`	`global_inicializada`
.bss	Variables globales sin inicializar (a cero)	`rw-`	`global_sin_inicializar`
Montón (heap)	Memoria dinámica: `malloc` y `free`, `new` en Java	`rw-`	el bloque que devuelve `malloc`
Pila (stack)	Un marco por llamada: locales, parámetros y dirección de retorno	`rw-`	`local`, `a`, `b`, `resultado`

Saltar al contenido principal

REGIÓN	CONTENIDO	PERMISOS	EN MEMORIA.C
Bibliotecas	Código y datos de libc y de otras bibliotecas cargadas	`r-x` y `rw-`	`printf` , `malloc`

memoria.c C

```c
#include <stdio.h>
#include <stdlib.h>

int global_inicializada = 42;     /* segmento .data */
int global_sin_inicializar;       /* segmento .bss  */

int suma(int a, int b) {          /* codigo en .text */
    int resultado = a + b;        /* variable local: pila */
    return resultado;
}

int main(void) {
    int local = 7;                          /* pila */
    int *dinamica = malloc(5 * sizeof(int)); /* monton (heap) */
    dinamica[0] = suma(local, global_inicializada);

    printf("codigo   (suma)          : %p\n", (void *)suma);
    printf("datos    (.data)         : %p\n", (void *)&global_inicializada);
    printf("bss      (.bss)          : %p\n", (void *)&global_sin_inicializar);
    printf("monton   (malloc)        : %p\n", (void *)dinamica);
    printf("pila     (variable local): %p\n", (void *)&local);
    printf("resultado = %d\n", dinamica[0]);

    free(dinamica);
    return 0;
}
```

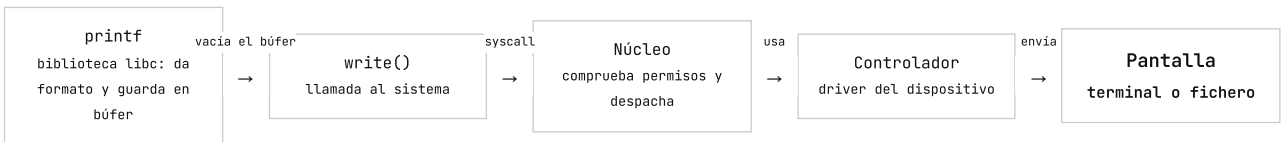
Terminal BASH

```
$ gcc -Wall -o memoria memoria.c
$ ./memoria
codigo   (suma)          : 0x5624f17831a9
datos    (.data)         : 0x5624f1786010
bss      (.bss)          : 0x5624f1786018
monton   (malloc)        : 0x5625018cd2a0
pila     (variable local): 0x7ffeb5d7b94c
resultado = 49
$ size memoria
   text    data     bss     dec     hex filename
   2167     628      12    2807     af7 memoria
```

CÓMO LEER LA SALIDA

- Código, datos y `.bss` quedan juntos (empiezan por `0x55…`), porque son partes del mismo fichero ejecutable.
- La pila está en direcciones mucho más altas (`0x7ff…`) y el montón en otra zona, lejos del ejecutable.
- Las direcciones cambian en cada ejecución: el sistema operativo aleatoriza el reparto (**ASLR**) para dificultar ataques.

Del printf a la pantalla: quién hace qué



CPU — CICLO CAPTAR-DECODIFICAR-EJECUTAR

La CPU lee la instrucción a la que apunta el **contador de programa**, la decodifica, la ejecuta en la ALU y avanza. Los registros son su memoria más rápida.

Saltar al contenido principal

MEMORIA — VIRTUAL Y FÍSICA

Cada proceso ve direcciones **virtuales**. La MMU las traduce a direcciones físicas por páginas, y así unos procesos no pueden leer la memoria de otros.

PERIFÉRICOS — SIEMPRE MEDIANTE EL NÚCLEO

Disco, teclado, pantalla o red se manejan con controladores del kernel. Cuando un dispositivo termina, avisa a la CPU con una **interrupción**.

strace BASH

```
$ strace -e trace=write ./memoria          # salida a la terminal: 6 llamadas write
write(1, "codigo   (suma)          : 0x557"..., 42) = 42
write(1, "datos    (.data)         : 0x557"..., 42) = 42
write(1, "bss      (.bss)          : 0x557"..., 42) = 42
write(1, "monton   (malloc)        : 0x557"..., 42) = 42
write(1, "pila     (variable local): 0x7ff"..., 42) = 42
write(1, "resultado = 49\n", 15) = 15
$ strace -e trace=write ./memoria > /dev/null   # salida a un fichero: 1 sola llamada
write(1, "codigo   (suma)          : 0x562"..., 225) = 225
```

/proc/PID/maps BASH

```
$ ./espera &
pid=1563
$ grep -E 'espera|heap|stack' /proc/1563/maps
55647215d000-55647215e000 r--p ... /tmp/espera
55647215e000-55647215f000 r-xp ... /tmp/espera      # codigo (.text)
556472161000-556472162000 rw-p ... /tmp/espera      # datos (.data y .bss)
556486b59000-556486b7a000 rw-p ... [heap]           # monton
7ffeb556e000-7ffeb5593000 rw-p ... [stack]          # pila
```

PROGRAMA, PROCESO E HILO

- **Programa**: el fichero en disco. **Proceso**: una instancia de ese programa en ejecución, con su espacio de direcciones. **Hilo**: un flujo de ejecución dentro de un proceso.
- Los hilos de un mismo proceso comparten código, datos y montón, pero cada uno tiene su propia pila y sus propios registros. Por eso dos hilos pueden estropear los mismos datos, y por eso dos procesos distintos no pueden hacerlo sin permiso del núcleo.

Del doble clic a la ejecución, paso a paso

Los cuatro niveles anteriores describen quién participa. Esta secuencia describe **en qué orden** ocurre todo cuando se ejecuta un programa, tanto si lo lanzas con doble clic como si escribes `./programa` en una terminal.

1. **Localización y comprobaciones.** El sistema operativo localiza el fichero, verifica que existe, que tienes permiso para ejecutarlo y que su formato es uno que sabe cargar (ELF en Linux, PE en Windows).
2. **Creación del proceso.** El cargador crea un proceso, le asigna un espacio de direcciones virtual y coloca en él el código, los datos y una pila inicial. Si el programa necesita bibliotecas dinámicas, las carga también.
3. **Preparación del entorno de ejecución.** Si existe un *runtime* (JVM, CLR, intérprete de Python), se inicia primero y es él quien carga tu programa. Ofrece servicios como gestión de memoria, excepciones e hilos.
4. **Ejecución.** La CPU ejecuta las instrucciones una tras otra. El sistema operativo reparte el tiempo de procesador entre los procesos y arbitra el acceso a memoria, ficheros, red y dispositivos.
5. **Acceso a recursos.** Cuando el programa necesita algo fuera de la CPU y la RAM, lo pide mediante una API o una llamada al sistema; nunca manipula el hardware por su cuenta.
6. **Finalización.** Al terminar, el sistema operativo recupera la memoria, cierra los recursos abiertos y devuelve un código de salida a quien lo lanzó.

Arquitectura y compatibilidad

Un ejecutable **nativo** está construido para una combinación concreta de arquitectura y sistema operativo, por ejemplo x86-64 con Windows o ARM64 con Linux. Por eso un `.exe` de Windows no arranca en Linux, y un binario compilado para un procesador ARM no sirve en uno x86-64.

Saltar al contenido principal

El código intermedio y las máquinas virtuales (bloque 1.4) mejoran la portabilidad, aunque siguen necesitando un *runtime* compatible instalado en cada equipo. Los contenedores empaquetan la aplicación con sus dependencias de espacio de usuario, pero **comparten el núcleo** del sistema anfitrión y no son máquinas virtuales completas. Estas tres formas de «virtualizar» se comparan al final del bloque 1.4.

CONSECUENCIA PRÁCTICA

Cuando alguien dice «en mi equipo funciona», casi siempre hay una diferencia de arquitectura, de sistema operativo, de versión del *runtime* o de bibliotecas. Saber qué necesita un programa para ejecutarse es el primer paso para reproducir un fallo.

COMPRUEBA LO QUE HAS ENTENDIDO

Un compañero te pasa el fichero `informe` compilado en su portátil con ARM y Linux, y en tu ordenador con x86-64 y Linux no arranca. Explica la causa con los términos de este bloque y di dos soluciones posibles.

ERROR FRECUENTE — PILA Y MONTÓN NO SON LO MISMO

La pila guarda locales y crece y decrece sola con cada llamada. El montón guarda lo que pides tú con `malloc` (o `new` en Java) y hay que liberarlo (`free`) o lo libera un recolector. Una variable local que apunta al montón está en la pila, pero **el dato al que apunta** está en el montón.

AMPLIACIÓN — MALLOC GRANDE Y MMAP

Con glibc, los bloques de 128 KiB o más se piden al núcleo con `mmap` y no aparecen en la línea `[heap]` de `/proc/PID/maps` . Si quieres ver el montón, reserva pocos bytes, como en `espera.c` .

LABORATORIO 1 · UN PROGRAMA POR DENTRO (SESIÓN S2 · 65 MIN · PAREJAS)

1. Compila `gcc -Wall -o memoria memoria.c` y ejecútalo **tres veces**. Anota en una tabla las direcciones de código, datos, `.bss` , montón y pila. ¿Cuáles cambian entre ejecuciones? ¿Qué región tiene las direcciones más altas?
2. Ejecuta `size memoria` y relaciona cada columna (`text` , `data` , `bss`) con las variables y funciones del programa. ¿Dónde están `local` y `dinamica` ?
3. Ejecuta `strace -e trace=write,brk ./memoria` y después `strace -e trace=write ./memoria > /dev/null` . Cuenta las llamadas `write` en cada caso y explica la diferencia.
4. Compila `espera.c` , lánzalo en segundo plano (`./espera &`) y lee `/proc/PID/maps` mientras duerme. Localiza la región `r-xp` (código), `rw-p` (datos), `[heap]` y `[stack]` . ¿Por qué el código no es escribible?
5. Redacta un informe de una página con la tabla del paso 1 y tres respuestas: por qué el programa necesita al núcleo para imprimir, por qué cambian las direcciones y qué componente (CPU, memoria o periférico) interviene en cada paso de un `printf` .

Entrega: informe en PDF o Markdown en la tarea *Lab 1* de Moodle. **Antes de ejecutar** (actividad A1): dibuja en papel cómo crees que se reparte la memoria y compáralo al final con `/proc/PID/maps` .

Saltar al contenido principal

1.2

Fases del desarrollo de software

De la idea al programa funcionando y después: qué produce cada fase y cómo se organizan según el modelo.

CE1.b — Fases de desarrollo de una aplicación

FUNDAMENTO — CICLO DE VIDA DEL SOFTWARE

El **ciclo de vida** es el conjunto de fases por las que pasa una aplicación desde que surge la necesidad hasta que se retira. La norma ISO/IEC/IEEE 12207 (edición de 2026, que sustituye a la de 2017) describe los procesos de ese ciclo. Las fases son las mismas en todos los proyectos; lo que cambia según el modelo es cuántas veces y en qué orden se recorren. El RA1 termina en la **puesta en funcionamiento**, pero el mantenimiento cierra el círculo.

Por qué seguir un proceso

Construir software sin un proceso compartido funciona mientras el proyecto es pequeño y lo hace una sola persona. En cuanto participan varias, o el producto debe durar, las decisiones informales dejan de bastar. Un proceso común permite alinear las expectativas con quien necesita el producto, descubrir ambigüedades antes de que se conviertan en código, repartir responsabilidades y coordinar dependencias, comprobar la calidad con criterios explícitos, conservar las decisiones para facilitar el mantenimiento y aprender de cada entrega.

Saltarse pasos no elimina el trabajo: lo desplaza a un momento en el que corregir resulta más caro. Un requisito mal entendido cuesta minutos de conversación durante el análisis, horas de reescritura durante la codificación y días de incidencias y explicaciones una vez en producción. Por otra parte, tener fases no significa recorrerlas una sola vez y en orden: en los enfoques iterativos se recorren varias veces con distinto nivel de detalle.

La calidad como objetivo de todo el ciclo

Una aplicación de calidad no es solo una aplicación sin errores. Es una que satisface necesidades y mantiene un conjunto de atributos. El modelo de calidad de producto de la familia ISO/IEC 25000, en su edición de 2023 (ISO/IEC 25010), organiza esos atributos en nueve características.

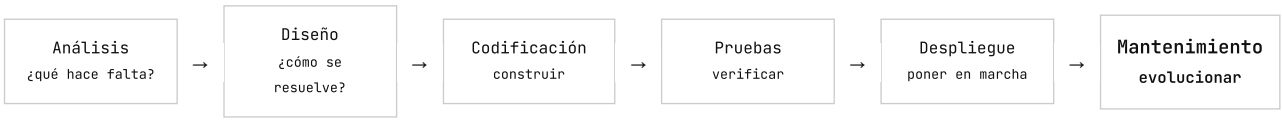
CARACTERÍSTICA	QUÉ PREGUNTA RESPONDE	EJEMPLO EN UNA APLICACIÓN DE TAREAS
Adecuación funcional	¿Hace lo que debe, y de forma correcta y completa?	Marca la tarea como hecha y respeta las fechas
Eficiencia de desempeño	¿Usa tiempo, memoria y red de forma razonable?	Lista 10 000 tareas en menos de 2 segundos
Compatibilidad	¿Convive con otros sistemas y funciona en los entornos previstos?	Se ejecuta en Windows y en Linux y exporta a un formato estándar
Capacidad de interacción	¿Puede utilizarse y comprenderse por su audiencia, incluida la accesibilidad?	Mensajes claros y uso completo con teclado
Fiabilidad	¿Mantiene un comportamiento correcto ante fallos previsibles?	No pierde datos si se cierra a mitad de un guardado
Seguridad	¿Protege la confidencialidad, la integridad y la disponibilidad?	Valida entradas y no guarda datos personales innecesarios
Mantenibilidad	¿Es fácil de comprender, probar y modificar?	Código legible con pruebas automáticas
Flexibilidad	¿Se adapta, instala y migra a otros entornos?	Se traslada a otro servidor con un solo comando
Protección (safety)	¿Evita causar daños a personas o a su entorno?	Relevante en sistemas médicos o industriales

Estos atributos entran a veces en tensión: hacer un programa más rápido puede volverlo más difícil de mantener, y hacerlo más seguro puede volverlo menos cómodo. Una decisión profesional hace visibles esos compromisos en lugar de prometer que todo será máximo a la vez, y cada fase del ciclo de vida contribuye a alguno de ellos.

Saltar al contenido principal

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Un programa produce siempre el resultado correcto, pero pierde los datos al cerrarse y solo su autor sabe modificarlo. ¿Es un producto de calidad? Razona la respuesta con al menos tres características de la tabla.

Las seis fases



FASE	ARTEFACTOS QUE PRODUCE	HERRAMIENTAS TÍPICAS	CRITERIO PARA PASAR A LA SIGUIENTE
Análisis	Requisitos funcionales y no funcionales, historias de usuario, casos de uso	Entrevistas, tablero de tareas, diagramas de casos de uso	Requisitos acordados y priorizados con el cliente
Diseño	Arquitectura, modelo de datos, diagramas UML, contratos de API	Herramientas de diagramas, OpenAPI	El diseño cubre todos los requisitos y es revisable
Codificación	Código fuente e historial de versiones	IDE, compilador, control de versiones	Código compilado y revisado por otra persona
Pruebas	Casos de prueba, informes, defectos registrados	Marcos de pruebas, análisis estático	Pruebas superadas y defectos críticos cerrados
Despliegue	Paquete versionado (`.jar` , imagen), notas de versión	Tuberías de integración y entrega continuas	La aplicación funciona en el entorno real
Mantenimiento	Incidencias, parches, nuevas versiones	Sistema de incidencias, monitorización	Se repite el ciclo para cada cambio

Cada fase por dentro

La tabla de arriba resume qué produce cada fase. Lo que sigue explica qué se hace realmente en ellas y por qué, porque el criterio CE1.b pide identificar las fases **y** sus tareas y artefactos.

Planificación y viabilidad

Antes de analizar nada hay que decidir si el proyecto merece la pena y con qué límites. La planificación define el problema, las personas interesadas, el alcance, los recursos, las restricciones, los riesgos y el criterio de éxito. Un buen alcance dice también lo que **queda fuera**.

La **viabilidad** se estudia desde varios ángulos: la *técnica* (conocimientos, herramientas e infraestructura disponibles), la *económica* (coste frente a beneficio y coste de oportunidad), la *operativa* (si encaja en el trabajo real de las personas), la *legal y ética* (protección de datos, propiedad intelectual, licencias, posibles daños) y la *temporal* (si es posible entregar dentro del plazo).

Una estimación no es una promesa exacta: es una previsión con supuestos e incertidumbre. Conviene dividir el trabajo en partes pequeñas, identificar dependencias y revisar la estimación conforme se aprende.

Análisis y requisitos

El análisis responde a la pregunta «¿qué necesita el sistema?». Se entrevista, se observa, se modela, se prioriza y se valida. Su resultado son los **requisitos**. Un requisito es una necesidad o condición verificable, y para serlo debe ser claro, necesario, viable, consistente, trazable y comprobable.

TIPO DE REQUISITO	QUÉ EXPRESA	EJEMPLO
Funcional	Un comportamiento o servicio del sistema	«El sistema permitirá marcar una tarea como completada»
No funcional (atributo de calidad)	Un nivel de servicio o una restricción medible	«El listado responderá en menos de 2 s con 10 000 tareas»
De dominio o de negocio	Una regla propia del contexto	«Solo el profesorado puede cerrar una evaluación»
	Una integración, tecnología, norma o formato obligatorio	«Los datos se exportarán en CSV con codificación UTF-8»

Saltar al contenido principal

Un requisito como «la aplicación será intuitiva» no se puede comprobar; «una persona nueva creará su primera tarea en menos de un minuto sin ayuda» sí. Las **historias de usuario** ayudan a conversar sobre lo que se necesita con la forma «Como [rol], quiero [acción] para [beneficio]». No están completas sin **criterios de aceptación**, que fijan cuándo se da la historia por buena.

Criterio de aceptación en formato Dado-Cuando-Entonces — GHERKIN

```
Dado que existen tareas de Programación y de Bases de Datos
Cuando filtro por "Programación"
Entonces solo aparecen las tareas de ese módulo
Y se mantiene visible el número de resultados
```

> VERIFICACIÓN Y VALIDACIÓN APLICADAS AL ANÁLISIS
>
> Un requisito puede estar bien redactado y ser el equivocado. Por eso el análisis termina con una **validación** con quien usa el producto: se construye lo que hace falta, no solo lo que se pidió.

Diseño

El diseño transforma el «qué» del análisis en un «cómo» suficientemente claro para poder construir y probar. Incluye la arquitectura y el reparto en componentes, el modelo de datos y la persistencia, las interfaces entre componentes y con sistemas externos, la interfaz y la experiencia de usuario, el tratamiento de errores, la seguridad y la observabilidad, y las decisiones sobre tecnología, despliegue y pruebas.

Diseñar no es producir diagramas decorativos. Un buen modelo responde a una pregunta concreta de su audiencia y se mantiene al nivel de detalle necesario; un diagrama que nadie consulta ni actualiza es un coste sin beneficio.

Implementación

En la implementación se escribe, integra y revisa el código. Las prácticas que la hacen sostenible son: nombres expresivos y estructura simple, **cohesión** alta (cada pieza hace una cosa) y **acoplamiento** controlado (cada pieza depende poco de las demás), ausencia de duplicación injustificada, validación de entradas y gestión explícita de errores, pruebas automatizadas, revisión por otras personas, *commits* pequeños y trazables, y documentación de las decisiones que no son evidentes.

«Código limpio» no es una estética universal: es código que el equipo puede leer, verificar y cambiar con seguridad.

Pruebas

Las pruebas responden a dos preguntas distintas (verificación y validación, ver más abajo) y se organizan por niveles según lo que comprueban.

NIVEL O PROPÓSITO	QUÉ COMPRUEBA	EJEMPLO
Unitaria	Una unidad pequeña, aislada del resto	Validar que una fecha existe
De integración	La colaboración entre componentes	El servicio de tareas y el almacén de datos
De sistema	El producto completo	El flujo crear, listar y completar una tarea
De aceptación	El valor y los criterios acordados con quien usa el producto	El filtro que pidió el profesorado
De regresión	Que un cambio no rompe lo que ya funcionaba	Repetir toda la batería tras añadir el guardado
De rendimiento	Tiempos, capacidad y estabilidad	Listar 10 000 tareas
De seguridad	Resistencia frente a amenazas	Entradas manipuladas y permisos
De usabilidad y accesibilidad	Uso eficaz por la audiencia prevista	Uso con teclado, mensajes y contraste

Dos ideas conviene tener claras. Primera: las pruebas revelan defectos presentes, pero **no demuestran por sí solas su ausencia**. Segunda: un caso de prueba bien definido incluye sus precondiciones, los datos de entrada, los pasos, el resultado esperado y el resultado obtenido; sin resultado esperado no hay prueba, solo una ejecución.

Despliegue y puesta en funcionamiento

Desplegar es llevar una versión al entorno donde se usará, con su configuración, sus datos y las instrucciones necesarias. Existen varias estrategias, con distinto equilibrio entre sencillez y riesgo.

Saltar al contenido principal

ESTRATEGIA	CÓMO FUNCIONA	VENTAJA Y RIESGO
Sustitución directa	Se detiene la versión antigua y se instala la nueva	Sencilla, pero hay un corte y cualquier fallo afecta a todos
Gradual (por fases)	La nueva versión se instala por grupos de equipos o de usuarios	Limita el alcance de un fallo, pero conviven dos versiones
Blue-green	Existen dos entornos idénticos; el tráfico cambia del activo al nuevo en un momento	Permite volver atrás casi al instante, a costa de duplicar infraestructura
Canary	Una pequeña parte del tráfico usa la nueva versión antes de extenderla a todos	Detecta problemas con pocos usuarios afectados; exige buena monitorización

Incluso en un proyecto pequeño conviene saber qué versión se instala, qué requisitos tiene, cuáles son los pasos de instalación y la configuración, y **cómo volver atrás** si algo falla.

Operación y retirada

Una vez en marcha, el software se opera y se observa: registros, métricas y avisos permiten detectar problemas antes que las personas usuarias. Cuando el producto llega al final de su vida útil, su **retirada** exige migrar o conservar los datos, comunicar las fechas, revocar las credenciales, cerrar los servicios y eliminar la información según las obligaciones que apliquen. Retirar mal un sistema es una fuente clásica de incidentes.

ACTIVIDADES TRANSVERSALES

- **Documentación:** se produce durante todo el ciclo, no al final.
- **Gestión de la configuración:** controla versiones, cambios y entornos (Git es su herramienta habitual).
- **Seguridad y calidad:** se integran en cada fase; no son una prueba final.
- **Comunicación y gestión de riesgos:** mantienen alineado al equipo y a quien usa el producto.

COMPRUEBA LO QUE HAS ENTENDIDO

Elige un requisito de la tabla de trazabilidad y explica en cuál de las ocho fases del ciclo se define, en cuál se diseña la solución, en cuál se comprueba y en cuál se puede ver afectado por el mantenimiento.

Modelos de ciclo de vida

MODELO	CÓMO RECORRE LAS FASES	CUÁNDO ENCAJA
Cascada	Una sola vez y en orden; cada fase cierra sus documentos	Requisitos estables, certificación o contrato cerrado
En V	Cascada donde cada fase de diseño tiene su nivel de pruebas	Sistemas críticos (medicina, automoción)
Iterativo e incremental	Varias vueltas; cada una añade funcionalidad	Los requisitos se aclaran con el uso
Espiral	Vueltas dirigidas por el análisis de riesgos	Proyectos grandes e inciertos
Ágil	Ciclos de 1 a 4 semanas; cada uno recorre todas las fases para poco alcance	Cambio frecuente y cliente disponible

Los modelos, explicados

Un **modelo de ciclo de vida** organiza el trabajo y las transiciones entre fases. Una **metodología** concreta además las prácticas, las responsabilidades, los artefactos y las reglas. Un **marco de trabajo**, como Scrum, define un contenedor mínimo que el equipo completa con sus propias técnicas de ingeniería. En la práctica, las organizaciones combinan enfoques.

Cascada

El modelo en cascada ordena el trabajo en grandes fases secuenciales, y el resultado aprobado de una es la entrada de la siguiente. Sus ventajas son unos hitos y una documentación claros; encaja cuando los requisitos y la tecnología son estables o la regulación exige evidencias formales.

Saltar al contenido principal

Sus riesgos son que el software utilizable llega tarde, que un error temprano puede propagarse hasta el final y que los cambios resultan caros. No debe describirse como «sin retorno posible»: los proyectos reales gestionan cambios. La diferencia está en cuánto cuesta volver atrás y en cómo se formaliza.

Modelo en V

Es una cascada en la que cada nivel de especificación o diseño se relaciona con una actividad de prueba. Su valor es que obliga a **planificar la verificación desde el principio**, y es útil cuando la trazabilidad y la evidencia son esenciales.

LADO DEL DESARROLLO	COMPROBACIÓN ASOCIADA
Necesidades y requisitos de usuario	`Pruebas de aceptación`
Requisitos del sistema	`Pruebas de sistema`
Arquitectura	`Pruebas de integración`
Diseño detallado	`Pruebas unitarias`

Prototipos

Un prototipo reduce la incertidumbre y obtiene retroalimentación temprana sobre una idea. El peligro principal es confundir una demostración rápida con software listo para producción.

TIPO DE PROTOTIPO	IDEA	CUIDADO
`Desechable`	Se construye para aprender y se descarta	Hay que descartarlo de verdad
`Evolutivo`	Se mejora hasta integrarse en el producto	Revisar arquitectura, seguridad y pruebas antes
`Horizontal`	Explora muchas pantallas con poca lógica	Puede dar una falsa sensación de avance
`Vertical`	Recorre una funcionalidad de extremo a extremo	Deja sin explorar el conjunto

Incremental, iterativo y espiral

Incremental significa entregar partes funcionales sucesivas, de modo que cada incremento amplía el producto. **Iterativo** significa revisar y mejorar lo ya construido a medida que aumenta el conocimiento. Son ideas distintas y pueden darse a la vez: un proyecto puede ser iterativo e incremental. El modelo **en espiral** organiza el trabajo en ciclos guiados por los riesgos: objetivos, análisis de riesgos, desarrollo y verificación, y planificación del ciclo siguiente.

> COMPRUEBA LO QUE HAS ENTENDIDO
> ¿Qué diferencia hay entre una iteración y un incremento? Pon un ejemplo de una aplicación de tareas en el que se produzcan a la vez.

Tipos de mantenimiento

Correctivo

Arregla un defecto que ya está en producción. Ejemplo: la aplicación se cierra al subir un fichero vacío.

Evolutivo

Añade funcionalidad nueva. Ejemplo: permitir pagar con un nuevo método.

Adaptativo

Adapta el software a un cambio del entorno sin cambiar lo que hace. Ejemplo: nueva versión del sistema operativo o de una biblioteca.

Saltar al contenido principal

Perfectivo

Mejora rendimiento o mantenibilidad sin cambiar el comportamiento. Ejemplo: reescribir una consulta lenta o reorganizar el código.

CINCO NOMBRES PARA CUATRO IDEAS

- La norma ISO/IEC 14764 clasifica el mantenimiento en **correctivo**, **adaptativo**, **perfectivo** y **preventivo**. El **preventivo** modifica el software para evitar fallos futuros antes de que ocurran. Ejemplo: sustituir una biblioteca sin soporte o añadir una comprobación de entradas vacías.
- En la práctica se usa también **evolutivo** para el cambio que añade funcionalidad (en la norma queda dentro del perfectivo). Conviene saber en cuál de los dos vocabularios habla cada equipo.

PUESTA EN FUNCIONAMIENTO

- El resultado del desarrollo es un **artefacto versionado** (fichero `.jar`, imagen de contenedor, paquete) que no cambia entre entornos.
- Recorre entornos de desarrollo, pruebas, preproducción y producción; el mismo artefacto sube de uno a otro.
- Incluye migrar datos y configuración, publicar, comprobar que funciona (**monitorización**) y tener un plan de vuelta atrás (*rollback*).

Traza de un ticket real — TEXTO

```
PROJ-482  "Como alumno quiero recuperar mi contraseña por correo"
  Análisis       -> criterios de aceptación y caso de uso
  Diseño         -> secuencia: Cliente > API > Servicio de correo > Base de datos
  Codificación   -> rama feature/PROJ-482, commits pequeños con mensaje claro
  Pruebas        -> prueba unitaria TokenServiceTest + prueba de aceptación
  Despliegue     -> merge a main > tubería > entorno de pruebas > producción
  Mantenimiento  -> PROJ-510 "el correo tarda 40 s" (perfectivo o correctivo)
```

VERIFICAR NO ES VALIDAR

- **Verificar** responde a «¿estamos construyendo bien el producto?»: comprueba la conformidad con la especificación (revisiones, análisis estático, pruebas unitarias).
- **Validar** responde a «¿estamos construyendo el producto correcto?»: comprueba que resuelve la necesidad real (pruebas de aceptación, demostración al cliente).
- Un programa puede pasar todas las pruebas y aun así no servir: la especificación estaba equivocada.

AMPLIACIÓN — EL CICLO COMPLETO TIENE OCHO ETAPAS

- Las seis fases del RA1 se centran en construir y poner en marcha. El ciclo de vida completo añade al principio la **planificación** (por qué y con qué recursos) y al final la **operación** y la **retirada** (apagar el sistema, migrar o borrar los datos y avisar a quien lo usa).
- Un dato personal que sigue guardado años después de retirar el servicio es un incidente esperando a ocurrir: retirar también es diseñar.

Trazabilidad: de la necesidad a la evidencia

Un requisito bien escrito se puede comprobar. La **trazabilidad** enlaza cada necesidad con el requisito, el diseño, la prueba y la evidencia, en los dos sentidos.

NECESIDAD	REQUISITO VERIFICABLE	DISEÑO	PRUEBA	EVIDENCIA
No olvidar entregas	RF-03 El sistema mostrará las entregas con fecha en los próximos 7 días, ordenadas por fecha	Consulta filtrada por fecha y vista ordenada	Prueba de aceptación con tres entregas en fechas distintas	Informe de la prueba PT-03 superada
Que vaya rápido	RNF-01 El listado responderá en menos de 2 s con 10 000 tareas	Índice por fecha y paginación	Prueba de carga con 10 000 tareas	Tiempo medido en el equipo de referencia
No perder datos al cerrar	RNF-02 Tras cerrar y abrir la aplicación, las tareas siguen ahí	Guardado en fichero o base de datos	Crear tareas, cerrar, abrir y comparar	Registro de la prueba PT-07

Saltar al contenido principal

ERROR FRECUENTE — PRUEBAS Y DEPURACIÓN NO SON LO MISMO

Probar es comprobar que el software cumple lo pedido y detectar fallos. **Depurar** es localizar y corregir la causa de un fallo ya detectado. Otro error habitual: creer que ágil significa no diseñar ni documentar; en realidad se hace poco y a menudo, en cada iteración.

ACTIVIDAD A4 · TRAZA DE UNA HISTORIA DE USUARIO (S2 · EQUIPOS DE 3)

Tu equipo recibe una historia (consultar calificaciones desde el móvil, exportar el calendario de exámenes a PDF o pedir cita para una tutoría). Rellena una tabla con las seis fases: **artefacto** que se produce, **quién** lo hace, **herramienta** y **criterio** para pasar a la siguiente. Después inventa un ejemplo de mantenimiento correctivo, evolutivo, adaptativo y perfectivo para esa aplicación y, si llegas, uno preventivo.

ACTIVIDAD A15 · TRAZABILIDAD DE TRES NECESIDADES (OPCIONAL · AMPLIACIÓN Y RECUPERACIÓN DE CE1.B)

Copia la tabla de trazabilidad de arriba y complétala para tres necesidades de tu historia de A4. Cada fila debe llevar un **requisito verificable** (sin «intuitivo» ni «rápido»), el elemento de diseño que lo cumple, la prueba y la evidencia que la demuestra. Extensión: añade en cada fila una amenaza (qué podría salir mal) y un control que la reduzca.

Saltar al contenido principal

1.3

Código fuente, objeto y ejecutable

La cadena de compilación completa en C: qué produce cada herramienta y por qué el objeto no se puede ejecutar.

CE1.c — Código fuente, objeto y ejecutable

> FUNDAMENTO — TRES FORMAS DEL MISMO PROGRAMA
>
> El **código fuente** es el texto que escribe la persona. El **código objeto** es la traducción a instrucciones máquina de un fichero fuente, pero incompleta: las referencias a funciones de otros ficheros o bibliotecas quedan sin resolver. El **ejecutable** es el binario final, con todos los símbolos resueltos y un punto de entrada, que el sistema operativo puede cargar. Entre ellos hay una cadena de herramientas: preprocesador, compilador, ensamblador y enlazador.

Quién traduce qué

Convertir texto legible en un programa que se ejecuta requiere una cadena de herramientas, y cada una tiene un nombre y una función precisos. Confundirlas es una de las fuentes de error más comunes en este criterio.

HERRAMIENTA	QUÉ HACE	EJEMPLO
Traductor	Transforma un programa de un lenguaje a otro; es el término general	Cualquiera de los siguientes
Compilador	Traduce una unidad o un programa completo **antes** de ejecutarlo y genera código objeto, intermedio o nativo	`gcc` , `javac`
Intérprete	Ejecuta el programa a partir de una representación suya; puede analizar, transformar y optimizar internamente	`python3` , `bash`
Ensamblador	Convierte lenguaje ensamblador en código máquina reubicable	`as`
Transpilador	Compila entre lenguajes de nivel similar	TypeScript a JavaScript (`tsc`)
Enlazador (linker)	Combina objetos y bibliotecas, resuelve símbolos y produce un ejecutable o una biblioteca	`ld`
Cargador (loader)	Sitúa el programa en memoria y prepara su ejecución	El del sistema operativo
JIT	Compila durante la ejecución, usando información del programa real	HotSpot, CLR
AOT	Compila antes de ejecutar; reduce el arranque a cambio de otras restricciones	GraalVM Native Image

Las herramientas anteriores no son excluyentes: un mismo entorno puede usar varias a la vez. `gcc` , por ejemplo, actúa como controlador que invoca al preprocesador, al compilador, al ensamblador y al enlazador.

Fuente, objeto, intermedio y ejecutable

Esta tabla resume las formas que puede adoptar un programa y sirve de referencia para el resto del bloque.

ARTEFACTO	¿LEGIBLE POR PERSONAS?	¿EJECUTABLE POR LA CPU?	EJEMPLO
Código fuente	Sí	No	`.c` , `.java` , `.cs` , `.py`
Código objeto nativo	No	Todavía no, no como programa completo	`.o` , `.obj`
Código intermedio	No, normalmente	No; necesita un *runtime* o una compilación posterior	Bytecode de la JVM, CIL
Ejecutable nativo	No	Sí, con los servicios del sistema operativo	ELF, PE (`.exe`)
Ensamblado de .NET	No (metadatos, CIL y recursos)	Necesita .NET, salvo con AOT	`.dll` , `.exe` administrado

Saltar al contenido principal

Dos matices evitan errores. El «código objeto» **no siempre significa** unos y ceros listos para la CPU: contiene código reubicable y referencias pendientes de resolver. Y no todo `.exe` contiene exclusivamente código nativo: uno de .NET contiene CIL y metadatos.

saludo.c C

```c
#include <stdio.h>
#define VERSION 3

int main(void) {
    printf("Version %d del programa\n", VERSION);
    return 0;
}
```

Recorre la cadena paso a paso (clic para ver más ejemplos)

1 Preprocesado | 2 Compilación | 3 Ensamblado | 4 Enlazado | 5 Carga

#include <stdio.h> gcc -E = +800 líneas

El preprocesador copia literalmente el contenido de stdio.h: saludo.c (6 líneas) se convierte en saludo.i con 820 líneas.

Clic para ver el siguiente ejemplo (1/2)

Los ficheros de la cadena, con tamaños reales

FICHERO	QUÉ ES	TAMAÑO	¿SE PUEDE EJECUTAR?
saludo.c	Código **fuente**: texto legible por personas	121 B	No
saludo.i	Fuente preprocesado (texto)	21 334 B	No
saludo.s	Ensamblador (texto)	713 B	No
saludo.o	Código **objeto**: binario reubicable con símbolos sin resolver	1 520 B	No
saludo	**Ejecutable** enlazado dinámicamente	15 960 B	Sí
saludo_static	Ejecutable enlazado estáticamente (`gcc -static`)	785 312 B	Sí, sin depender de libc.so

Terminal BASH

```bash
$ gcc -E saludo.c -o saludo.i && wc -l saludo.i
820 saludo.i
$ gcc -S saludo.i -o saludo.s
$ gcc -c saludo.s -o saludo.o
$ file saludo.o
saludo.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
$ nm saludo.o
0000000000000000 T main
                 U printf
$ readelf -h saludo.o | grep Type
  Type:                              REL (Relocatable file)
$ gcc saludo.o -o saludo
$ nm saludo | grep -E ' main| printf'
0000000000001149 T main
                 U printf@GLIBC_2.2.5
$ readelf -h saludo | grep Type
  Type:                              DYN (Position-Independent Executable file)
$ ldd saludo
        linux-vdso.so.1 (0x00007f6cad985000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f6cad600000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f6cad987000)
$ ./saludo
Version 3 del programa
```

Saltar al contenido principal

saludo.s (simplificado) X86-64

```
    .text
    .section    .rodata
.LC0:
    .string    "Version %d del programa\n"
    .text
    .globl    main
main:
.LFB0:
    pushq    %rbp
    movq    %rsp, %rbp
    movl    $3, %esi
    leaq    .LC0(%rip), %rax
    movq    %rax, %rdi
    movl    $0, %eax
    call    printf@PLT
    movl    $0, %eax
    popq    %rbp
    ret
```

Enlace estático (.a)

El enlazador **copia** el código de las bibliotecas dentro del ejecutable. El fichero es grande (785 312 B frente a 15 960 B) y autosuficiente, pero para actualizar la biblioteca hay que volver a compilar.

Enlace dinámico (.so)

El ejecutable solo guarda el **nombre** de la biblioteca y `ld.so` la carga al arrancar. Ahorra disco y memoria y permite parchear la biblioteca una vez, pero falla si falta o cambia de versión.

Y EN LOS LENGUAJES INTERPRETADOS

Un programa Python o un script de shell tiene **fuente**, pero no genera objeto ni ejecutable persistentes: el intérprete lo lee y lo ejecuta. Java y C# generan un equivalente del objeto (`.class` , `.dll`) con código intermedio; se estudia en el bloque 1.4.

DEPURACIÓN Y PUBLICACIÓN: EL MISMO FUENTE, DOS COMPILACIONES

- `gcc -g -O0 saludo.c` deja la información de depuración (`with debug_info` en `file`) y no optimiza: sirve para seguir el código paso a paso con un depurador. En este caso ocupa 17 136 B.
- `gcc -O2 -s saludo.c` optimiza y elimina los símbolos (`stripped`): es lo que se entrega. Ocupa 14 472 B. El código de máquina apenas cambia en un programa tan corto; con programas mayores la diferencia crece.
- El modo **depuración** (*debug*) y el modo **publicación** (*release*) existen en todos los lenguajes y herramientas. Java y .NET tienen sus propios equivalentes.

¿En qué fase falla? Los mensajes lo dicen

MENSAJE TÍPICO	FASE DONDE FALLA	CAUSA
`fatal error: stdoi.h: No such file or directory`	Preprocesado	Fichero de cabecera mal escrito o inexistente
`error: expected ',' or ';' before 'return'`	Compilación	Error de sintaxis en el código fuente
`undefined reference to` sqrt`	Enlazado	Falta enlazar la biblioteca (`-lm`) o definir la función. Solo aparece si el argumento no es constante: con `sqrt(2.0)` gcc calcula el resultado al compilar
`error while loading shared libraries: libfoo.so`	Carga	El cargador no encuentra una biblioteca dinámica
`Segmentation fault`	Ejecución	El proceso accede a memoria que no le pertenece

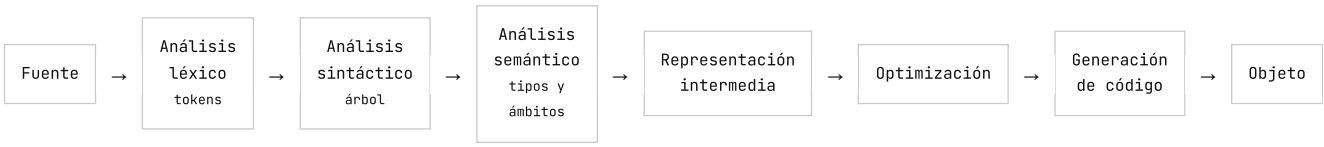
Saltar al contenido principal

Léxico, sintáctico, semántico y de ejecución

TIPO DE ERROR	EJEMPLO REAL	MENSAJE	QUIÉN LO DETECTA
Léxico	`int x = 3 @ 2;` en C	`error: stray '@' in program`	Analizador léxico del compilador
Sintáctico	Falta un `;` al final de la línea	`error: expected ',' or ';' before 'return'` (gcc) o `';' expected` (javac)	Analizador sintáctico
Semántico	`int x = "hola";` en Java	`error: incompatible types: String cannot be converted to int`	Análisis semántico (tipos)
De ejecución	Acceder fuera de un array, dividir entre cero	`Segmentation fault` o una excepción	El proceso en marcha, no el compilador

Las fases de un compilador

Un compilador no traduce de una sola vez. Divide el trabajo en fases, cada una con una responsabilidad distinta, y esas fases explican por qué existen los distintos tipos de error.



- **Léxico:** agrupa los caracteres en unidades llamadas *tokens* (palabras clave, identificadores, números, símbolos).
- **Sintáctico:** comprueba que los tokens siguen la gramática del lenguaje y construye un árbol.
- **Semántico:** valida los tipos, los nombres, los ámbitos y las reglas que dependen del contexto.
- **Representación intermedia:** describe el programa de forma independiente del texto original y del procesador de destino.
- **Optimización:** transforma el programa sin cambiar el comportamiento observable que el lenguaje permite.
- **Generación de código:** produce las instrucciones para el destino, que se guardan como código objeto.
- **Enlace:** resuelve las referencias entre módulos y bibliotecas para crear el ejecutable.

Las implementaciones reales pueden fusionar, repetir o reordenar estas tareas; el modelo sirve para entender, no para describir un compilador concreto.

Errores según el momento en que aparecen

Saber **cuándo** se detecta un error orienta la búsqueda de su causa. Un mensaje de compilación se corrige mirando el fuente; uno de enlace, mirando qué módulos y bibliotecas se han pasado; uno de carga, mirando el entorno; uno de ejecución, mirando los datos; y uno de lógica exige pruebas, porque nadie avisa.

MOMENTO	EJEMPLOS DE ERROR	QUIÉN LO DETECTA
Compilación	Token inválido, sintaxis incorrecta, tipo incompatible	El compilador
Enlace	Símbolo definido en otro módulo que no se encuentra	El enlazador
Carga	Biblioteca o *runtime* ausente o incompatible	El cargador
Ejecución	División por cero, fichero inexistente, acceso a un valor nulo	El propio programa en marcha
Lógica	Resultado incorrecto aunque no haya ninguna excepción	Las pruebas o las personas usuarias

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Un programa compila sin avisos, se enlaza y arranca, pero al calcular la media de un array vacío devuelve un valor absurdo sin lanzar ninguna excepción. ¿En qué fila de la tabla está ese error y por qué ninguna herramienta lo ha detectado antes?

Saltar al contenido principal

NOTA — PRINTF EN EL ENSAMBLADOR

Con `printf` sin argumentos de formato, `gcc` puede sustituirla por `puts`. Con `%d` mantiene `printf`, como en `saludo.c`. Tu salida de `-S` puede variar ligeramente con la versión de `gcc`.

ERROR FRECUENTE — OBJETO NO ES ORIENTADO A OBJETOS

Código objeto es la salida binaria del compilador para un fichero; no tiene relación con la programación orientada a objetos. Y compilar no basta para tener un ejecutable: sin el paso de enlazado, `saludo.o` no puede lanzarse.

LABORATORIO 2 · PARTE A: CADENA DE COMPILACIÓN EN C (SESIÓN S4 · 50 MIN · PAREJAS)

1. Ejecuta `gcc -E saludo.c -o saludo.i`, cuenta las líneas con `wc -l` y mira el final con `tail`. ¿Qué pasó con `VERSION`?
2. Ejecuta `gcc -S saludo.i -o saludo.s` y localiza `.LC0` y la instrucción `call`.
3. Ejecuta `gcc -c saludo.s -o saludo.o`. Con `file`, `nm` y `readelf -h saludo.o | grep Type` comprueba de qué tipo es y qué símbolos quedan sin resolver. **Antes de ejecutar `nm` ** (actividad A5) predice qué mostrará.
4. Enlaza con `gcc saludo.o -o saludo`, ejecuta `./saludo` y repite `file`, `nm` y `ldd`. ¿Por qué `printf` sigue apareciendo como `U`?
5. Compila `gcc -static saludo.c -o saludo_static` y compara el tamaño y la salida de `ldd`.
6. Rellena la tabla de los cinco ficheros (nombre, texto o binario, tamaño, qué contiene, si se puede ejecutar).

Entrega: un ZIP con `saludo.i`, `saludo.s`, `saludo.o`, `saludo` y el informe, en la tarea *Lab 2* de Moodle (la entrega se completa con las partes B y C del bloque 1.4).

Saltar al contenido principal

1.4

Código intermedio y máquinas virtuales

Bytecode, JVM, CLR y CPython: cómo se ejecuta un programa que no se compila directamente a código nativo.

CE1.d — Código intermedio y máquinas virtuales

> FUNDAMENTO — PORTABILIDAD CON CÓDIGO INTERMEDIO
>
> Un ejecutable nativo solo sirve para una arquitectura y un sistema operativo. Para escribir una vez y ejecutar en muchos sitios, algunos lenguajes compilan a un **código intermedio** (*bytecode*) que ejecuta una **máquina virtual** instalada en cada plataforma. El código intermedio es igual en todas partes; la máquina virtual es la que conoce el hardware real.

Por qué existe el código intermedio

Un compilador nativo tiene que conocer la arquitectura de destino. Si quieres que un mismo programa funcione en varios sistemas, hay dos opciones: compilarlo una vez por plataforma, o compilarlo **una sola vez a un formato neutro** y dejar que cada plataforma lo entienda con su propia máquina virtual. Los lenguajes con código intermedio eligen la segunda.

Esa decisión tiene ventajas claras: **portabilidad** (el mismo fichero `.class` o `.dll` sirve en cualquier sistema con el *runtime* adecuado), **seguridad** (la máquina virtual puede verificar el código antes de ejecutarlo y limitar lo que hace) y **optimización adaptativa** (el JIT usa información del programa real, algo que un compilador previo no tiene). También tiene un coste: un arranque más lento, más memoria consumida y la obligación de tener el *runtime* instalado.

Qué contiene una máquina virtual de proceso

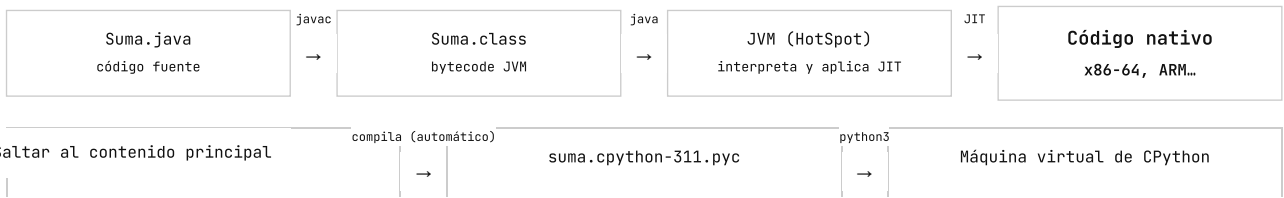
Una **máquina virtual de proceso** ofrece una plataforma abstracta a un programa concreto. No simula un ordenador entero: proporciona el conjunto de servicios que ese programa necesita para ejecutarse. Aunque la JVM y el CLR son productos distintos, sus piezas equivalen.

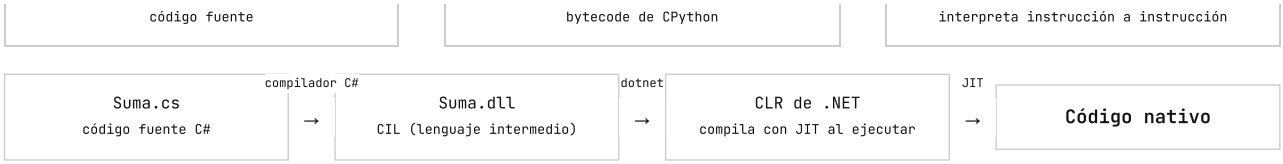
PIEZA	QUÉ HACE	EN LA JVM	EN EL CLR DE .NET
Cargador	Localiza y carga en memoria las clases o ensamblados que el programa necesita	Cargador de clases	Cargador de ensamblados
Verificador	Comprueba que el código intermedio es coherente antes de ejecutarlo	Verificador de bytecode	Verificación de CIL y metadatos
Ejecución	Interpreta el código intermedio y lo compila a nativo cuando conviene	Intérprete + JIT (C1 y C2)	Compilador JIT
Memoria	Reserva y organiza pila y montón; libera lo que ya no se usa	Áreas de memoria y recolector de basura	Montón administrado y recolector de basura
Servicios	Excepciones, hilos y acceso a la biblioteca base	Bibliotecas de la plataforma Java	Bibliotecas base de .NET

Por eso «Java es interpretado» o «Java es compilado» son ambas verdades a medias: el compilador `javac` traduce el fuente a bytecode, y la JVM lo interpreta y compila a nativo mientras se ejecuta.

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Un programa Java se ejecuta igual en Windows y en Linux, pero un programa en C compilado en Windows no arranca en Linux. Explica dónde está la diferencia usando los términos «código intermedio» y «máquina virtual».

Tres caminos hasta la ejecución





Suma.java JAVA

```java
public class Suma {
    public static int suma(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int resultado = suma(3, 4);
        System.out.println("Resultado: " + resultado);
    }
}
```

Terminal · bytecode de Java BASH

```
$ javac Suma.java && java Suma
Resultado: 7
$ od -An -tx1 -N8 Suma.class
 ca fe ba be 00 00 00 41
$ javap -c Suma
Compiled from "Suma.java"
public class Suma {
  public Suma();
    Code:
       0: aload_0
       1: invokespecial #1     // Method java/lang/Object."<init>":()V
       4: return

  public static int suma(int, int);
    Code:
       0: iload_0
       1: iload_1
       2: iadd
       3: ireturn

  public static void main(java.lang.String[]);
    Code:
       0: iconst_3
       1: iconst_4
       2: invokestatic  #7     // Method suma:(II)I
       5: istore_1
       6: getstatic     #13    // Field java/lang/System.out:Ljava/io/PrintStream;
       9: iload_1
      10: invokedynamic #19,  0 // InvokeDynamic #0:makeConcatWithConstants:(I)Ljava/lang/String;
      15: invokevirtual #23    // Method java/io/PrintStream.println:(Ljava/lang/String;)V
      18: return
}
```

CÓMO LEERLO

- `ca fe ba be` es el número mágico de todo fichero `.class`. Los cuatro bytes siguientes son la versión: `00 00 00 41` es la versión mayor 65, que corresponde a Java 21 (Java 25 es la 69).
- `iload_0` y `iload_1` cargan los parámetros en la **pila de operandos**, `iadd` suma los dos de la cima y `ireturn` devuelve el resultado.
- El bytecode es el mismo en Windows, Linux o macOS. Lo que cambia es la JVM.

Comparativa de plataformas con código intermedio

PLATAFORMA	CÓDIGO INTERMEDIO	FICHERO	QUIÉN LO EJECUTA	¿GENERA CÓDIGO NATIVO?
Java, Kotlin	Bytecode JVM	`.class`, `.jar`	JVM (HotSpot)	Sí, con JIT (niveles C1 y C2)
.NET (C#)	CIL	`.dll`, `.exe`	CLR	Sí, con JIT (y opciones AOT)

Saltar al contenido principal

PLATAFORMA	CÓDIGO INTERMEDIO	FICHERO	QUIÉN LO EJECUTA	¿GENERA CÓDIGO NATIVO?
Python (CPython)	Bytecode de CPython	`.pyc`	Intérprete de CPython	Por defecto no; el JIT es experimental y no viene activado en 3.14
JavaScript (V8)	Bytecode interno	en memoria	Motor V8	Sí, con JIT
WebAssembly	Bytecode Wasm	`.wasm`	Motor del navegador o de servidor	Sí, lo compila el motor

El compilador JIT: de interpretar a compilar lo que se usa mucho

La JVM empieza **interpretando** el bytecode y cuenta cuántas veces se ejecuta cada método y cada bucle. Cuando algo se repite lo suficiente (código «caliente»), el compilador **JIT** lo traduce a código nativo: primero con el nivel 3 (compilador C1, rápido, con perfilado) y después con el nivel 4 (C2, más lento pero muy optimizado). Es lo que hace que un programa Java tarde algo en arrancar y luego corra casi como uno nativo.

Terminal · PrintCompilation — BASH

```
$ java -XX:+PrintCompilation Bucle | grep 'Bucle::trabajo'
     33    7 %     3       Bucle::trabajo @ 4 (25 bytes)
     33    8       3       Bucle::trabajo (25 bytes)
     34    9 %     4       Bucle::trabajo @ 4 (25 bytes)
     36    7 %     3       Bucle::trabajo @ 4 (25 bytes)   made not entrant
```

> CÓMO LEERLO
>
> El primer número es el tiempo en ms; la cifra `3` o `4` es el nivel de compilación; `%` indica que se sustituye un bucle mientras se ejecuta; `made not entrant` marca una versión ya sustituida por otra mejor.

MEDIDA (EQUIPO DE PRUEBAS, JDK 21)	RESULTADO	LECTURA
`java Bucle` (JIT activo)	1 250 ms	El bucle caliente se compila a código nativo
`java -Xint Bucle` (solo intérprete)	12 139 ms	Unas 10 veces más lento sin JIT
Arranque de un «hola»: C, Python y Java	2 ms, 14 ms y 47 ms	La JVM paga un coste de arranque que después compensa

> IMPORTANTE
>
> Estas cifras se midieron en un equipo concreto con JDK 21. En el tuyo saldrán otras, pero la proporción entre JIT e intérprete y entre arranques se mantendrá.

Python también compila a bytecode

suma.py — PYTHON

```python
def suma(a, b):
    return a + b

resultado = suma(3, 4)
print("Resultado:", resultado)
```

Saltar al contenido principal

Terminal · bytecode de Python BASH

```
$ python3 -m py_compile suma.py && ls __pycache__
suma.cpython-311.pyc
$ od -An -tx1 -N4 __pycache__/suma.cpython-311.pyc
 a7 0d 0d 0a
$ python3
>>> import dis
>>> def suma(a, b):
...     return a + b
>>> dis.dis(suma)
  3           0 RESUME                   0

  4           2 LOAD_FAST                0 (a)
              4 LOAD_FAST                1 (b)
              6 BINARY_OP                0 (+)
             10 RETURN_VALUE
```

NOTA — DIS CAMBIA ENTRE VERSIONES

El listado de `dis` depende de la versión de Python: en 3.11 empieza por `RESUME` y versiones posteriores usan instrucciones especializadas. Fíjate en la idea (cargar operandos en la pila, operar, devolver), no en los nombres exactos. Lo mismo vale para el número mágico del `.pyc` y para su nombre: las capturas de arriba son de Python 3.11 (`cpython-311`); si tu equipo tiene Python 3.12 verás `suma.cpython-312.pyc` y otro número mágico. Comprueba tu versión con `python3 --version` .

Más allá del JIT: compilación anticipada (AOT)

Project Leyden (OpenJDK)

Guarda en una **caché AOT** lo que la JVM aprende en una ejecución de entrenamiento (clases cargadas y enlazadas, perfiles de métodos) para arrancar antes. Empezó con la carga y el enlace anticipados de clases en el JDK 24 y añade nuevas piezas en versiones posteriores. Sigue siendo Java con JVM.

GraalVM Native Image

Compila el programa Java completo a un **ejecutable nativo** mediante análisis estático. Arranca en milisegundos y usa poca memoria, a cambio de tiempos de compilación largos y de tener que declarar lo dinámico (reflexión). Ya no hay JVM en ejecución.

ERROR FRECUENTE — LA JVM NO ES VIRTUALBOX

Una máquina virtual de **proceso** (JVM, CLR, la de CPython) ejecuta un programa concreto con código intermedio. Una máquina virtual de **sistema** (VirtualBox, KVM) simula un ordenador entero para instalar un sistema operativo. Comparten el nombre, no el objetivo. Y «Java es interpretado» es solo verdad a medias: la JVM interpreta al principio y compila a nativo lo que se ejecuta mucho.

Tres cosas que se llaman «virtual»

	MÁQUINA VIRTUAL DE PROCESO	MÁQUINA VIRTUAL DE SISTEMA	CONTENEDOR
Ejemplo	JVM, CLR, CPython	VirtualBox, KVM	Docker, Podman
Qué aísla	Un programa con código intermedio	Un ordenador completo con su núcleo	Procesos y dependencias de usuario
Núcleo	El del sistema anfitrión	Uno propio dentro de la máquina virtual	Comparte el del anfitrión
Coste	Arranque de la máquina virtual	Alto: memoria y disco de un sistema entero	Bajo: arranca como un proceso

Saltar al contenido principal

AMPLIACIÓN — QUÉ HAY DENTRO DE UNA .DLL DE .NET (CASO PARALELO, NO SE EJECUTA EN EL AULA)

- `dotnet build` compila C# a un ensamblado (`.dll` o `.exe`) que contiene **CIL** y **metadatos** (tipos, miembros, referencias). Es el equivalente del `.class` de Java.
- El CLR carga el ensamblado y su compilador JIT lo traduce a código nativo al ejecutarlo. También existen la compilación anticipada de ensamblados y **Native AOT**, que produce un ejecutable nativo sin JIT (como GraalVM Native Image en el mundo Java).
- **.NET 10** es una versión LTS publicada el 11-11-2025 con soporte hasta el 14-11-2028. Las versiones 8 y 9 terminan su soporte el 10-11-2026.
- Esta pieza no forma parte de los laboratorios: no se ha ejecutado en las aulas. Se ofrece al alumnado con interés como ampliación.

Caso paralelo: C# y .NET

MATERIAL DE AMPLIACIÓN

Esta pieza muestra que la idea del bloque no es exclusiva de Java. **No se ejecuta en los laboratorios**: sirve para reconocer la misma arquitectura en otra plataforma.

C# es un lenguaje de propósito general, multiparadigma y de tipado estático. **.NET** es la plataforma: incluye el *runtime* (CLR), las bibliotecas base, el SDK, las herramientas y los marcos de aplicación. No conviene confundirlos: otros lenguajes pueden ejecutarse sobre .NET, y C# necesita una implementación y unas bibliotecas para producir aplicaciones útiles.

El camino es análogo al de Java. **Roslyn** analiza el código C# y lo compila a CIL con metadatos. El resultado se guarda en un **ensamblado**, normalmente un `.dll` ; la extensión no significa que sea una biblioteca nativa de Windows. El CLR carga, verifica y ejecuta el ensamblado, y su JIT transforma cada método en instrucciones nativas cuando se necesita. En los escenarios compatibles, **Native AOT** produce código nativo durante la publicación y elimina el JIT.

Terminal · flujo básico con .NET — BASH

```
$ dotnet new console -n Saludo     # crea el proyecto (Saludo.csproj y Program.cs)
$ cd Saludo
$ dotnet build                     # compila a CIL: bin/Debug/net10.0/Saludo.dll
$ dotnet run                       # compila si hace falta y ejecuta
$ dotnet publish -c Release        # deja en publish/ lo necesario para desplegar
```

Los tres comandos hacen cosas distintas: `build` **compila**, `run` compila si hace falta y **ejecuta**, y `publish` **prepara para desplegar**. Un despliegue *framework-dependent* necesita que el equipo de destino tenga .NET instalado; uno *self-contained* incluye el *runtime* en el paquete a costa de un tamaño mayor. El directorio `obj/` guarda resultados intermedios y `bin/` el ensamblado y sus dependencias.

ASPECTO	JAVA (JDK)	C# (.NET)
Fuente	`.java`	`.cs`
Compilador habitual	`javac`	Roslyn (dentro del SDK de .NET)
Código intermedio	Bytecode de la JVM	CIL (con metadatos)
Unidad empaquetada	`.class` y `.jar`	Ensamblado: `.dll` o `.exe`
Entorno de ejecución	JVM	CLR de .NET
Estrategias	JIT; AOT con GraalVM Native Image o Project Leyden	JIT; AOT de ensamblados y Native AOT

LABORATORIO 2 · PARTE B: BYTECODE DE JAVA Y PYTHON (SESIÓN S4 · 45 MIN · PAREJAS)

1. Compila `javac Suma.java` y ejecuta `java Suma` . Con `od -An -tx1 -N8 Suma.class` localiza `ca fe ba be` y decide qué versión de Java compiló el fichero (versión mayor = versión de Java + 44).
2. Con `javap -c Suma` localiza las cuatro instrucciones de `suma` y explica qué hace cada una con la pila.
3. Genera el `.pyc` con `python3 -m py_compile suma.py` , localízalo en `__pycache__` y desensambla la función con `dis` .
4. Rellena una tabla de tres columnas (C, Java, Python): qué se genera, qué lo ejecuta y si es portable entre arquitecturas.

Saltar al contenido principal

LABORATORIO 2 · PARTE C: JIT Y ARRANQUE (S4 · 15 MIN · PAREJAS)

1. Compila `Bucle.java` y mide `java Bucle` frente a `java -Xint Bucle` (esta puede tardar más de 10 segundos).
2. Ejecuta `java -XX:+PrintCompilation Bucle | grep trabajo` y localiza los niveles 3 y 4.
3. Mide el arranque de `hola` en C, Java y Python con `time` (varias veces cada uno). Escribe una conclusión de dos frases.

Entrega: ZIP con `Suma.class`, `__pycache__/suma*.pyc` y el informe con las tres tablas, en la tarea *Lab 2* de Moodle.

LABORATORIO 2 · PARTE D (OPCIONAL): DEPURACIÓN FRENTE A PUBLICACIÓN (S4 · AMPLIACIÓN · PAREJAS)

1. Compila `gcc -g -O0 saludo.c -o saludo_dbg` y `gcc -O2 -s saludo.c -o saludo_rel`. Compara tamaños con `ls -l` y la salida de `file` (busca `with debug_info` y `stripped`).
2. Ejecuta `size` sobre ambos. ¿Qué cambia y qué no? ¿Por qué el código apenas varía en un programa de seis líneas?
3. Genera el ensamblador de cada versión con `gcc -S -O0` y `gcc -S -O2` y cuenta las líneas con `wc -l`.
4. Inventario final: clasifica cada fichero del laboratorio como fuente, intermedio, objeto o ejecutable y anota qué herramienta lo generó.

No puntúa aparte: si se entrega, se valora dentro del criterio «cadena de compilación en C» de la rúbrica del Lab 2.

Saltar al contenido principal

1.5

Lenguajes de programación

Cómo clasificar un lenguaje con criterio: nivel, ejecución, paradigma y tipos.

CE1.e — Clasificación de los lenguajes de programación

> FUNDAMENTO — CLASIFICAR ES CRUZAR EJES
>
> Ninguna etiqueta sola describe un lenguaje. Se cruzan varios **ejes independientes**: nivel de abstracción, estrategia de ejecución, paradigma y sistema de tipos. Un mismo lenguaje ocupa una casilla en cada eje: Java es de alto nivel, se compila a bytecode, es orientado a objetos y de tipado estático fuerte.

Qué define a un lenguaje

Un lenguaje de programación es un lenguaje formal que permite expresar algoritmos y estructuras de datos. Para describirlo con precisión conviene separar sus componentes.

- **Léxico:** los caracteres y los *tokens* válidos (palabras clave, identificadores, números, operadores).
- **Sintaxis:** las reglas para combinar tokens en instrucciones y programas bien formados.
- **Semántica:** el significado de esas construcciones.
- **Sistema de tipos:** las reglas sobre los valores y las operaciones permitidas entre ellos.
- **Modelo de ejecución:** cómo se transforma y ejecuta el programa (compilación, interpretación, máquina virtual).
- **Ecosistema:** bibliotecas, herramientas y comunidad. No forman estrictamente el lenguaje, pero condicionan su uso real.

Estos componentes explican los errores del bloque 1.3. Un error **léxico** contiene un símbolo o token no reconocido; uno **sintáctico** rompe la gramática; uno **semántico** está bien formado, pero carece de significado válido en el contexto (por ejemplo, asignar un texto a un entero); y uno **lógico** compila y se ejecuta, pero produce un resultado incorrecto.

Los cuatro ejes (cambia de pestaña)

Abstracción | Ejecución | Paradigma | Tipado

Nivel 1	Lenguaje máquina Instrucciones binarias de una arquitectura concreta.	10111000 00000001…	1GL
Nivel 2	Ensamblador Mnemónicos que equivalen uno a uno a instrucciones máquina.	mov eax, 1	2GL
Nivel 3	Bajo nivel Sintaxis de alto nivel con control manual de la memoria.	C, Rust, C++	3GL
Nivel 4	Alto nivel Abstracciones potentes y gestión automática de memoria.	Java, Python, Kotlin	3GL

Ocho lenguajes, cuatro ejes

LENGUAJE	NIVEL	EJECUCIÓN	PARADIGMA	TIPADO
C	Bajo	Nativo (gcc)	Imperativo	Estático, débil
Java	Alto	Bytecode + JVM con JIT	Orientado a objetos	Estático, fuerte
Python	Alto	Bytecode en CPython	Multiparadigma	Dinámico, fuerte
JavaScript	Alto	Motor con JIT (V8)	Multiparadigma, eventos	Dinámico, débil
Kotlin	Alto	Bytecode JVM (o nativo con Kotlin/Native)	Objetos y funcional	Estático, fuerte, con inferencia

Saltar al contenido principal

LENGUAJE	NIVEL	EJECUCIÓN	PARADIGMA	TIPADO
Rust	Bajo/medio, sin recolector	Nativo (LLVM)	Multiparadigma	Estático, fuerte
SQL	Muy alto	Lo interpreta el motor de la base de datos	Declarativo	Estático (tipos de columna)
Bash	Alto	Interpretado línea a línea	Imperativo	Sin tipos (todo son cadenas)

Fuerte o débil: la misma operación en tres lenguajes

Python (CPython 3.11) PYTHON

```
>>> 1 + "2"
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> "3" * 2
'33'
>>> "5" - 2
TypeError: unsupported operand type(s) for -: 'str' and 'int'
```

JavaScript (Node) JAVASCRIPT

```
> 1 + "2"
'12'
> "3" * 2
6
> "5" - 2
3
```

Java (JDK 21) JAVA

```
String s = 1 + "2";        // "12": concatenación de texto
int n = "3" * 2;           // no compila:
// error: bad operand types for binary operator '*'
```

CÓMO LEERLO

- Python no mezcla `int` con `str` al sumar y lanza error en ejecución; en cambio `"3" * 2` repite el texto. Es fuerte y dinámico.
- JavaScript convierte los tipos por su cuenta (`"3" * 2` da `6` y `1 + "2"` da `"12"`). Es débil y dinámico.
- Java es estático: el error de `"3" * 2` se detecta al **compilar**, antes de ejecutar nada.

Java (JDK 21) JAVA

```
var n = 3;          // el compilador deduce int
n = "hola";        // error: incompatible types:
                   // String cannot be converted to int
```

ERROR FRECUENTE — INFERIR EL TIPO NO ES TENER TIPADO DINÁMICO

- Con `var` (Java desde la versión 10), `let` o `auto` , el compilador **deduce** el tipo a partir del valor, pero ese tipo queda **fijo** para siempre. La variable sigue siendo estática y fuerte.
- En un lenguaje dinámico como Python, en cambio, la misma variable puede apuntar primero a un entero y luego a un texto. El tipo está en el valor, no en la variable.

Los paradigmas y su convivencia

Un paradigma es una forma de organizar el pensamiento al programar. La tabla resume los más relevantes.

PARADIGMA	IDEA	RASGOS HABITUALES
Imperativo	Describir cómo cambia el estado paso a paso	Asignación, bucles, secuencias
[illegible]	Organizar el trabajo en procedimientos reutilizables	Funciones y módulos

Saltar al contenido principal

PARADIGMA	IDEA	RASGOS HABITUALES
Orientado a objetos	Colaboración entre objetos con estado y comportamiento	Clases, encapsulación, herencia, polimorfismo
Funcional	Componer funciones y reducir el estado mutable	Inmutabilidad, funciones de orden superior
Declarativo	Expresar qué resultado se desea, no cómo obtenerlo	Consultas SQL, reglas
Lógico	Inferir respuestas a partir de hechos y reglas	Unificación y búsqueda con retroceso (Prolog)
Dirigido por eventos	Reaccionar a sucesos externos	Escuchadores y funciones de respuesta

La mayoría de lenguajes actuales son **multiparadigma**: Java, C#, Python y JavaScript admiten estilos orientados a objetos, funcionales, imperativos y dirigidos por eventos en distinta medida. Por eso preguntar «¿de qué paradigma es Python?» no tiene una respuesta única; sí la tiene preguntar qué estilo se está usando en un fragmento concreto.

> SOBRE LOS LENGUAJES DE «MEDIO NIVEL»
> «Lenguaje de medio nivel» es una etiqueta informal que se aplica a lenguajes como C porque combinan abstracciones de alto nivel con control directo de la memoria. No es una categoría normalizada: en los exámenes conviene clasificar por el eje de abstracción y justificar.

> CLASIFICACIÓN CLÁSICA POR GENERACIONES
> Muchos libros clasifican los lenguajes en **1GL** (máquina), **2GL** (ensamblador), **3GL** (alto nivel: C, Java, Python), **4GL** (orientados a datos o informes: SQL) y **5GL** (basados en restricciones o lógica: Prolog). Es útil conocerla, pero no sustituye a los ejes de arriba.

> HTML Y CSS NO SON LENGUAJES DE PROGRAMACIÓN
> HTML y CSS son lenguajes de **marcado** y de **estilo**: describen contenido y aspecto, no algoritmos. Se estudian en Lenguajes de marcas (0373). JavaScript sí es de programación.

> ERROR FRECUENTE — «COMPILADO» O «INTERPRETADO» NO DESCRIBE UN LENGUAJE
> Describe una **implementación**. Existen intérpretes de C y compiladores de Java a nativo (GraalVM). CPython compila a bytecode y por tanto «Python no compila» es falso. Para justificar una clasificación, cita siempre la implementación concreta que usas.

Dos lenguajes más: TypeScript y C#

TypeScript añade tipos opcionales a JavaScript y se **transpila** a JavaScript: el compilador `tsc` traduce entre dos lenguajes de nivel similar y después el motor de JavaScript ejecuta el resultado. Su tipado es *estático gradual*: se puede escribir código con y sin anotaciones. **C#** sigue el camino Roslyn, CIL y CLR descrito en el bloque 1.4; con `var total = 3;` el compilador infiere `int`, así que el tipado sigue siendo estático.

LENGUAJE	TIPADO HABITUAL	EJECUCIÓN TÍPICA	FORTALEZAS FRECUENTES
C	Estático	Nativo tras compilar y enlazar	Sistemas, control y eficiencia
Java	Estático	Bytecode sobre la JVM con JIT o AOT	Portabilidad y ecosistema empresarial
Python	Dinámico	Bytecode sobre el intérprete	Automatización, datos y rapidez de desarrollo
JavaScript	Dinámico	Motor con interpretación y JIT	Plataforma web y ecosistema muy amplio
TypeScript	Estático gradual	Transpilación a JavaScript	Mantenibilidad de aplicaciones JavaScript grandes
C#	Estático	CIL sobre .NET con JIT o AOT	Aplicaciones multiplataforma y herramientas integradas
Rust	Estático	Nativo	Seguridad de memoria sin recolector de basura

Observa que «Ejecución típica» habla de una implementación concreta: cada fila es un recorrido, no una propiedad absoluta del lenguaje.

Cómo elegir un lenguaje

No existe «el mejor lenguaje»: existe una decisión más o menos adecuada y una justificación verificable. Al elegir para un proyecto conviene valorar:

Saltar al contenido principal

…uncionales;
…estricciones de despliegue;

- el ecosistema, las bibliotecas y la madurez de las herramientas;
- la experiencia del equipo y la facilidad para formar o contratar;
- el rendimiento, la seguridad y la mantenibilidad;
- la interoperabilidad con el sistema existente;
- el soporte, la comunidad, la gobernanza y el ciclo de versiones;
- las licencias y el coste total, no solo el precio inicial.

Una justificación válida cita evidencias (una medición, una limitación documentada, una experiencia previa del equipo) y no se apoya solo en la popularidad.

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Clasifica C# por nivel de abstracción, tipado, paradigmas y ejecución. Después explica por qué decir que Python «no tiene tipos» es incorrecto.

> ACTIVIDAD A17 · CLASIFICADOR MULTIDIMENSIONAL (OPCIONAL · AMPLIACIÓN Y REFUERZO DE CE1.E)
>
> Elige tres lenguajes de la tabla y añade una columna con un **escenario adecuado** y otro **poco adecuado**, con una razón técnica (rendimiento, ecosistema, seguridad de tipos, tiempo de arranque). Regla del aula: no se admiten las etiquetas «compilado» o «interpretado» sin describir la implementación concreta.

> LABORATORIO 3 · PARTE A: TABLA DE CLASIFICACIÓN (SESIÓN S6 · 30 MIN · PAREJAS)
>
> 1. Clasifica **C, Java, Python, JavaScript, Kotlin, Rust, SQL y Bash** según nivel, ejecución, paradigma y tipado.
> 2. Para cada fila aporta una **evidencia**: un comando que hayas ejecutado (`file` , `javap` , `dis`) en los lenguajes instalados o un enlace a documentación oficial en el resto.
> 3. Marca como «discutible» cualquier etiqueta de tipado fuerte o débil que no puedas justificar con un ejemplo.
> 4. No uses «compilado» ni «interpretado» sin decir qué implementación concreta describes (por ejemplo: «Java: `javac` genera bytecode y HotSpot lo interpreta y compila con JIT»).
>
> **Entrega:** tabla con evidencias en el informe del Lab 3 (Moodle).

Saltar al contenido principal

1.6

Herramientas de desarrollo

Editor, IDE, terminal, dependencias, control de versiones, tableros e IA: qué hace cada una y cómo evaluarla.

CE1.f — Evaluar las herramientas de desarrollo

> FUNDAMENTO — EVALUAR UNA HERRAMIENTA CON CRITERIOS
>
> El criterio CE1.f pide **evaluar la funcionalidad**, no solo nombrar herramientas. Evaluar es contestar con evidencias a preguntas como: ¿qué problema resuelve?, ¿en qué fase interviene?, ¿qué cuesta usarla?, ¿qué alternativas hay? Las herramientas se agrupan en familias según su función.

Familias de herramientas del desarrollo de software

FAMILIA	QUÉ RESUELVE	EJEMPLOS	FASE DONDE INTERVIENE
Editor de código	Escribir texto con resaltado de sintaxis	Vim, Nano, Notepad++	Codificación
IDE	Editar, compilar, depurar, refactorizar y gestionar el proyecto en un solo entorno	IntelliJ IDEA, Eclipse, Visual Studio Code	Codificación y pruebas
Terminal	Ejecutar y automatizar con órdenes; es el entorno de las tuberías de CI	bash, zsh, PowerShell	Todas
Herramienta de construcción	Compilar, empaquetar y lanzar pruebas de forma repetible	make, Maven, Gradle, npm	Codificación y despliegue
Gestor de dependencias	Descargar y versionar bibliotecas y las que estas necesitan	pip, Maven, npm	Codificación
Depurador	Detener un programa en ejecución y examinar su estado	gdb, depurador del IDE	Pruebas
Control de versiones	Historial de cambios y trabajo en paralelo	Git, GitHub, GitLab	Todas
Tablero de tareas	Organizar el trabajo pendiente, en curso y terminado	GitHub Projects, Jira, Trello	Análisis y gestión
Asistente de IA	Sugerir, explicar o generar código y pruebas	GitHub Copilot, ChatGPT, Claude	Codificación y pruebas

Una cadena de herramientas, no una lista

Las herramientas del desarrollo no funcionan aisladas: forman una **cadena** en la que la salida de una es la entrada de la siguiente. El editor produce el código; el compilador o intérprete lo traduce; el gestor de dependencias aporta las bibliotecas; el depurador ayuda a entender los fallos; Git registra los cambios y la plataforma remota permite trabajar en equipo; la integración continua ejecuta todo automáticamente con cada cambio.

Además de las familias de la tabla anterior, una cadena moderna suele incluir estas otras.

HERRAMIENTA	FUNCIÓN	ENTRADA Y SALIDA
Linter o analizador estático	Detecta defectos, malas prácticas y reglas de estilo **sin ejecutar** el programa	Fuente → avisos
Marco de pruebas	Define, ejecuta e informa de las pruebas automáticas	Código y pruebas → informe
Plataforma remota	Aloja repositorios, incidencias, revisiones y automatizaciones	Commits → colaboración
Contenedor	Empaqueta la aplicación y sus dependencias de forma reproducible	Imagen → proceso aislado
Observabilidad	Recoge registros, métricas y trazas en operación	Sistema en marcha → señales

Evaluar una herramienta es, por tanto, decir **en qué punto de la cadena está**, qué recibe, qué produce, qué riesgo reduce y qué limitaciones tiene. «Es la más usada» no es una justificación técnica.

Saltar al contenido principal

Qué ofrece un IDE frente a un editor

NotaMedia.java — UD1 — Entorno de desarrollo integrado (maqueta)

NotaMedia.java | Terminal

```java
public static double notaMedia(int[] notas) {
    int suma = 0;
    int contador = 0;
    for (int i = 0; i ⩽ notas.length; i++) {
        if (notas[i] ⩾ 0 && notas[i] ⩽ 10) {
            suma += notas[i];
            contador++;
        }
    }
    return suma / contador;
}
```

Análisis en tiempo real

Un **servidor de lenguaje** (protocolo LSP) analiza el código mientras escribes y subraya errores antes de compilar, además de ofrecer autocompletado según el contexto.

Depurador integrado

Pones un **punto de ruptura**, ejecutas paso a paso y ves las variables y la pila de llamadas sin salir del editor.

Refactorización

Renombrar un método en 40 ficheros o extraer una función se hace de forma segura, porque el IDE entiende el código y no solo el texto.

Integración con Git

Muestra los cambios, permite confirmar, cambiar de rama y resolver conflictos desde la propia ventana.

Dependencias: las bibliotecas que traen otras bibliotecas

Terminal · pip BASH

```
$ python3 -m venv .venv
$ .venv/bin/pip install requests==2.32.3
$ .venv/bin/pip list
Package            Version
------------------ ---------
certifi            2026.7.22
charset-normalizer 3.5.1
idna               3.20
pip                24.0
requests           2.32.3
setuptools         79.0.1
urllib3            2.8.0
$ .venv/bin/pip show requests | grep Requires
Requires: certifi, charset-normalizer, idna, urllib3
```

CÓMO LEERLO

Pediste una sola biblioteca (`requests`) y el gestor instaló cinco paquetes: `Requires` lista las **dependencias directas** y esas a su vez pueden tener las suyas (**transitivas**). Fijar versiones (`requirements.txt` , `pom.xml` , `package-lock.json`) es lo que evita el clásico «en mi máquina funciona».

Saltar al contenido principal

Git por dentro: un commit es un objeto y una rama es un puntero

Terminal · Git BASH

```
$ git init && git add hola.py && git commit -m "Primer commit"
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 9b17ffd0bc4aa33f2921374490b59f3bd08896e8
author Alumno <alumno@ejemplo.es> 1789000000 +0200
committer Alumno <alumno@ejemplo.es> 1789000000 +0200

Primer commit
$ git cat-file -p HEAD:hola.py
print("hola")
$ cat .git/refs/heads/main
7a65eb1cf0d3cc91d35226fe98d7c1721dd9fe2b
```

> ERROR FRECUENTE — GIT NO ES GITHUB
>
> **Git** es el sistema de control de versiones distribuido que instalas en tu equipo. **GitHub** y **GitLab** son servicios que alojan repositorios Git y añaden petición de cambios, incidencias y automatización. Puedes usar Git sin ninguna cuenta en la nube.

Buenas prácticas con Git y con la colaboración

Git registra los cambios como *commits* en un repositorio distribuido. Trabajar bien con él es más una cuestión de hábitos que de comandos:

- **No incluir** contraseñas, claves ni datos personales; una vez en el historial, borrarlos del último *commit* no basta.
- Usar un fichero `.gitignore` para excluir los artefactos generados (por ejemplo, `bin/`, `obj/` o `__pycache__/`) y los secretos locales.
- Hacer *commits* pequeños, coherentes y con mensajes que expliquen el porqué del cambio.
- Revisar los cambios (`git status`, `git diff`) antes de confirmarlos.
- Trabajar con ramas breves y revisar los cambios mediante una petición de integración (*pull request* o *merge request*).
- Recordar que Git no sustituye a una estrategia de copia de seguridad ni a la gestión de permisos.

Terminal · flujo mínimo BASH

```
$ git init
$ git status
$ git add README.md
$ git commit -m "docs: define el alcance del gestor de tareas"
$ git switch -c feature/listado
$ git log --oneline --graph
```

Biblioteca, framework, runtime y SDK

PIEZA	QUÉ ES	QUIÉN LLAMA A QUIÉN	EJEMPLO EN JAVA
Biblioteca	Conjunto de funciones que tu programa usa	Tu código llama a la biblioteca	`java.lang.Math`, una biblioteca de utilidades
Framework	Estructura que dirige parte del flujo y llama a tu código	El framework llama a tu código (inversión de control)	JUnit llama a tus métodos de prueba; Spring Boot arranca la aplicación
Runtime	Lo necesario para ejecutar programas ya hechos	Ejecuta tu programa	JRE: la JVM y las bibliotecas base
SDK	Lo necesario para desarrollar: compilador, herramientas y runtime	Tú lo usas para construir	JDK: `javac`, `javap`, `jar` y el JRE

Integración, entrega y despliegue continuos

> INTEGRACIÓN CONTINUA
>
> Cada cambio se integra en la rama principal con frecuencia y dispara una construcción y unas pruebas automáticas. Detecta pronto que la integración se rompe.

Saltar al contenido principal

> ENTREGA CONTINUA
>
> Además, el software queda siempre en un estado publicable. La publicación real la decide una persona con un botón.

> DESPLIEGUE CONTINUO
>
> Todo cambio que supera la tubería se publica automáticamente, sin intervención humana. Exige pruebas muy fiables.

Contenedores

Un contenedor ayuda a reproducir el entorno en el que corre una aplicación. Conviene distinguir dos términos: la **imagen** es la plantilla inmutable, y el **contenedor** es una instancia de esa imagen en ejecución. El fichero que describe cómo se construye la imagen (el *Dockerfile*) debe versionarse junto con el código.

Buenas prácticas: usar imágenes mantenidas, fijar las versiones, ejecutar con un usuario sin privilegios siempre que sea posible y dejar los secretos fuera de la imagen. Recuerda que un contenedor **comparte el núcleo** del sistema anfitrión: aísla procesos y dependencias, pero no es una máquina virtual completa.

Herramientas CASE

CASE (*Computer-Aided Software Engineering*) es el nombre histórico de las herramientas que apoyan actividades de la ingeniería del software. Se suelen clasificar en tres grupos.

- **Upper CASE:** planificación, requisitos, análisis y diseño.
- **Lower CASE:** implementación, pruebas, documentación y mantenimiento.
- **Integrated CASE:** trazabilidad entre varias fases.

Hoy sus capacidades aparecen repartidas entre gestores de requisitos, herramientas de modelado, IDE, plataformas DevOps y generadores de documentación. La generación automática de código y la ingeniería inversa son útiles, pero sus resultados necesitan revisión humana.

RAD y plataformas low-code

ENFOQUE	QUÉ PRIORIZA	QUÉ HAY QUE EVALUAR
RAD (desarrollo rápido de aplicaciones)	Prototipos, reutilización, herramientas visuales y entregas rápidas con participación frecuente de quien usa el producto	Escalabilidad y mantenimiento a largo plazo
Low-code / no-code	Construir aplicaciones con componentes visuales y poca o ninguna programación	Dependencia del proveedor, límites de personalización, pruebas, seguridad, rendimiento y accesibilidad

Son adecuadas para ciertos procesos internos y para prototipos. La cautela es la misma que con cualquier herramienta: una aplicación creada sin escribir código sigue necesitando requisitos, pruebas, seguridad y mantenimiento.

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Un compañero propone construir el gestor de tareas de clase con una plataforma low-code. Escribe tres ventajas y tres riesgos, y decide con qué criterios de la matriz de evaluación los comprobarías.

Matriz para evaluar una herramienta

CRITERIO	QUÉ DEBES ESCRIBIR	EVIDENCIA ESPERABLE
Fase	En qué fase o fases del ciclo de vida interviene la herramienta	Referencia a la tabla de fases del bloque 1.2
Funcionalidad clave	Qué hace que otras herramientas no hagan o no tan bien	Captura o comando ejecutado
Evidencia	Qué has probado tú y qué has observado	Salida real, no copiada de un manual
Límites	Qué no resuelve, qué cuesta (curva de aprendizaje, licencia, recursos)	Un caso concreto donde falla
Alternativa	Otra herramienta profesional para lo mismo	Nombre y una diferencia

Saltar al contenido principal

CRITERIO	QUÉ DEBES ESCRIBIR	EVIDENCIA ESPERABLE
Veredicto 0-3	Puntuación con una frase de justificación	Coherente con lo anterior

VARIANTE PONDERADA PARA COMPARAR DOS IDE (OPCIONAL)

En lugar de puntuar de 0 a 3, reparte 100 puntos entre estos criterios, puntúa cada IDE en cada uno y multiplica. **Las puntuaciones sin evidencia no cuentan.**

CRITERIO PARA COMPARAR DOS IDE	PESO
Plataforma y lenguajes que soporta	20
Depuración y pruebas	20
Integración con Git	15
Seguridad de las extensiones	15
Accesibilidad	10
Consumo y rendimiento	10
Licencia y coste	10

La IA como herramienta del proceso

Qué aporta la IA

- Completa y explica código.
- Propone pruebas y documentación.
- Traduce entre lenguajes y sugiere refactorizaciones.

Qué exige de ti

- Entender y revisar cada línea antes de integrarla.
- Probarla con casos límite: vacío, cero, valores fuera de rango.
- No pegar contraseñas, datos personales ni código confidencial.

DATO DE CONTEXTO — LA IA EN EL DESARROLLO

En la encuesta de Stack Overflow de 2025 (unos 48 900 encuestados) el 84 % de los desarrolladores usa o planea usar herramientas de IA, frente al 76 % del año anterior, pero el 46 % desconfía de su precisión y dos de cada tres encuentran soluciones que casi funcionan pero fallan en algún detalle. La conclusión para el oficio: la IA acelera, pero la responsabilidad de que el código funcione sigue siendo tuya.

ERROR FRECUENTE — «COMPILA, LUEGO ESTÁ BIEN»

Que un código compile no demuestra que sea correcto. Los defectos que la IA introduce suelen ser silenciosos: límites de bucle, divisiones enteras, casos vacíos. Se detectan leyendo, ejecutando con datos límite y usando el depurador.

Saltar al contenido principal

NotaMedia.java (Laboratorio 3, parte C) JAVA

```java
public class NotaMedia {

    // Generado por un asistente de IA con el prompt:
    // "Escribe un metodo que devuelva la nota media de un array de notas (0-10),
    //  ignorando las notas no validas."
    public static double notaMedia(int[] notas) {
        int suma = 0;
        int contador = 0;
        for (int i = 0; i <= notas.length; i++) {
            if (notas[i] >= 0 && notas[i] <= 10) {
                suma += notas[i];
                contador++;
            }
        }
        return suma / contador;
    }

    public static void main(String[] args) {
        int[] notas = {7, 4, 9, 12, 5};
        System.out.println("Media: " + notaMedia(notas));
    }
}
```

LABORATORIO 3 · PARTE B: TALLER DE HERRAMIENTAS (SESIÓN S6 · 45 MIN · PAREJAS)

1. **Terminal:** con `ps -o pid,ppid,stat,cmd` , una tubería con `grep` y `man` localiza tu propio shell y explica qué hace cada orden.
2. **IDE:** abre `Suma.java` y `suma.py` en el editor que haya en el aula. Localiza cinco funciones: autocompletado, error subrayado, punto de ruptura, renombrado seguro e integración con control de versiones. Captura dos.
3. **Dependencias:** crea un entorno virtual, instala `requests==2.32.3` y obtén con `pip show requests` sus dependencias. Guarda el resultado con `pip freeze > requirements.txt` . (Plan B sin red: `pip install --no-index --find-links ruedas requests` .)
4. **Git:** crea un repositorio con un fichero, haz un commit y examina `git cat-file -t HEAD` , `git cat-file -p HEAD` y `cat .git/refs/heads/main` . ¿Qué es realmente una rama?
5. **Matriz:** evalúa las cuatro herramientas (terminal, IDE, gestor de dependencias y Git) con los seis criterios de la tabla y una puntuación de 0 a 3 razonada.

LABORATORIO 3 · PARTE C: REVISIÓN CRÍTICA DE CÓDIGO GENERADO POR IA (S6 · 30 MIN · PAREJAS)

Un asistente de IA ha escrito `NotaMedia.java` a partir del prompt que ves en el comentario. **No mires la solución.** Sigue este proceso:

1. Compílalo y ejecútalo con `notas = {7, 4, 9, 12, 5}` . Anota lo que ocurre y lee el mensaje de error completo.
2. Corrige el defecto y vuelve a ejecutar. Comprueba a mano el resultado esperado (la nota 12 no es válida y debe ignorarse).
3. Prueba con un array vacío y con un array de notas todas no válidas. Corrige lo que falle.
4. Redacta un informe corto: qué defectos había, **cómo** los detectaste (excepción, prueba manual, IDE) y cómo reescribirías el prompt para que la IA pidiera antes las condiciones de contorno.

Entrega: informe y `NotaMedia.java` corregido en la tarea *Lab 3* de Moodle. Se evalúa tu criterio, no que la IA se equivoque.

Saltar al contenido principal

1.7

Metodologías ágiles

Manifiesto, Scrum, Kanban y XP: características, ventajas y escenarios de uso frente al modelo secuencial.

CE1.g — Metodologías ágiles: características y escenarios

FUNDAMENTO — EL MANIFIESTO ÁGIL (2001)

Las metodologías ágiles nacen del **Manifiesto ágil**, que prefiere: **individuos e interacciones** sobre procesos y herramientas; **software funcionando** sobre documentación exhaustiva; **colaboración con el cliente** sobre negociación contractual; y **responder al cambio** sobre seguir un plan. Se concretan en doce principios y en marcos como Scrum, Kanban y XP. Frente a ellas, el modelo secuencial (cascada) sigue teniendo sentido cuando los requisitos son estables y hay que certificar.

De dónde vienen las metodologías ágiles

Los modelos secuenciales asumen que se pueden conocer los requisitos casi completos al principio. En muchos productos esto no se cumple: las necesidades cambian, el mercado responde y la tecnología evoluciona. Descubrir tarde que se construyó lo equivocado es caro. Las metodologías ágiles proponen **ciclos cortos** que entregan software utilizable y obtienen retroalimentación real para decidir el siguiente paso.

El Manifiesto ágil se apoya en doce principios; los más citados son satisfacer al cliente con entregas tempranas y continuas, acoger los cambios incluso tarde, entregar con frecuencia, colaborar a diario con el negocio, mantener un ritmo sostenible, cuidar la excelencia técnica y reflexionar periódicamente para mejorar. Los valores del manifiesto **no niegan** el valor de los procesos, la documentación, los contratos o los planes: afirman que los elementos de la izquierda se valoran más.

Cascada, Scrum, Kanban y XP

ASPECTO	CASCADA	SCRUM	KANBAN	XP
Unidad de trabajo	El proyecto completo	Historia dentro de un sprint	Tarjeta en el flujo	Historia o tarea pequeña
Cadencia	Una entrega final	Sprint de un mes o menos	Flujo continuo	Iteraciones cortas y entregas frecuentes
Roles	Jefe de proyecto, analistas, programadores…	Product Owner, Scrum Master y Developers	No prescribe roles	Cliente en el equipo y programadores
Artefactos	Documentos por fase	Product Backlog, Sprint Backlog e Incremento	Tablero con límites de WIP	Historias, pruebas y código compartido
Ante el cambio	Costoso una vez cerrados los requisitos	Se incorpora entre sprints	Se incorpora en cualquier momento	Acogido de forma continua

Scrum según la Scrum Guide 2020

SCRUM TEAM

- Un **Product Owner** que maximiza el valor y ordena el Product Backlog.
- Un **Scrum Master** que ayuda a entender y aplicar Scrum.
- Los **Developers**, que construyen el incremento.

EVENTOS

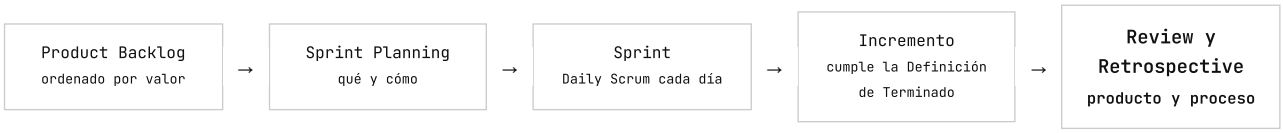
- **Sprint** de un mes o menos, contenedor de todo lo demás.
- **Sprint Planning**, **Daily Scrum** (15 minutos), **Sprint Review** y **Sprint Retrospective**.

Saltar al contenido principal

ARTEFACTOS Y COMPROMISOS

- **Product Backlog** con el Product Goal.
- **Sprint Backlog** con el Sprint Goal.
- **Incremento** con su Definición de Terminado.

Un ciclo de Scrum



Un tablero Kanban con límites de trabajo en curso (WIP)



POR QUÉ EL LÍMITE DE WIP

Limitar cuántas tarjetas puede haber a la vez en cada columna obliga a **terminar antes de empezar** y hace visible el cuello de botella: la columna que se llena mientras la siguiente se queda vacía. Se miden el **tiempo de ciclo** (desde que empieza una tarjeta hasta que termina) y el **rendimiento** (tarjetas terminadas por unidad de tiempo).

Qué mide Kanban y qué reglas hace explícitas

Kanban **visualiza el flujo** de trabajo, **limita el trabajo en curso**, hace explícitas las políticas del equipo (qué significa «terminado», cuándo se pasa de columna) y mejora a partir de datos. Además del tiempo de ciclo y del rendimiento ya vistos, se usa el **lead time**: el tiempo desde que se solicita un trabajo hasta que se entrega.

La diferencia entre *lead time* y tiempo de ciclo es que el primero incluye la espera antes de empezar, que es lo que percibe quien hizo la petición. Limitar el WIP ayuda a terminar antes de empezar más cosas y, con ello, a acortar ambos tiempos, y el rendimiento sirve para prever cuánto se puede entregar en el futuro.

Prácticas técnicas de Extreme Programming

Desarrollo dirigido por pruebas (TDD)

Se escribe primero la prueba que falla, luego el código mínimo para que pase y se refactoriza.

Programación en pareja

Dos personas, un teclado: una escribe y la otra revisa y piensa en voz alta.

Integración continua

Cada cambio se integra y se prueba de forma automática varias veces al día.

Refactorización

Mejorar el diseño del código sin cambiar lo que hace, con pruebas que lo garanticen.

y lo que no

Saltar al contenido principal

Scrum es un marco ligero para abordar problemas complejos. Su idea central es la **inspección y adaptación** frecuentes: en cada evento se mira qué se ha hecho y se ajusta el rumbo. El propósito de cada evento es el siguiente.

EVENTO	PROPÓSITO
Sprint Planning	Decidir por qué el Sprint es valioso, qué se puede hacer y cómo se hará
Daily Scrum	Inspeccionar el progreso hacia el Sprint Goal y adaptar el plan del día
Sprint Review	Inspeccionar el resultado con las personas interesadas y adaptar el Product Backlog
Sprint Retrospective	Mejorar la calidad y la eficacia del equipo

El **Increment** es el resultado utilizable de un Sprint, y solo cuenta como tal si cumple la **Definición de Terminado**, el estándar de calidad que el equipo acuerda. Un incremento «casi terminado» no es un incremento.

> LO QUE SCRUM NO PRESCRIBE
>
> Scrum no exige historias de usuario, puntos de historia, tableros ni sprints de dos semanas: son prácticas compatibles, no reglas del marco. Lo único fijo es el límite de un mes como máximo para el Sprint.

Extreme Programming: la ingeniería que sostiene la agilidad

XP refuerza la parte técnica con prácticas como la programación por parejas, el desarrollo dirigido por pruebas, la integración continua, el diseño simple, la refactorización, la propiedad colectiva del código, las entregas pequeñas y el ritmo sostenible. Una ceremonia ágil sin prácticas técnicas puede producir entregas frecuentes de baja calidad: la agilidad no está en las reuniones, sino en la capacidad de cambiar el software con seguridad.

¿Cuándo usar cada enfoque?

ESCENARIO	ENFOQUE RECOMENDABLE	POR QUÉ
Firmware de un dispositivo médico con certificación	Secuencial o en V	Requisitos cerrados y trazabilidad exigida por la normativa
Startup que valida una idea con usuarios reales	Ágil (Scrum o Kanban)	Requisitos inciertos y necesidad de comentarios rápidos
Equipo de soporte que recibe incidencias a diario	Kanban	Llegan a cualquier hora y no encajan en sprints
Migración con fecha y alcance fijados por contrato	Híbrido	Plan global fijo con iteraciones y revisiones internas
Proyecto de clase de cuatro semanas	Scrum simplificado	Sprints de una semana, revisión y retrospectiva semanales

Cómo decidir entre enfoques

La tabla de escenarios no es una receta. La elección no debe basarse en la moda, sino en factores como la **incertidumbre** de los requisitos, el **riesgo**, la **regulación**, el tamaño y la distribución del equipo, la frecuencia de entrega deseada y la capacidad real de recibir retroalimentación. En la práctica es habitual combinar enfoques: por ejemplo, un plan global por hitos con iteraciones internas.

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Justifica un enfoque para un gestor de tareas de clase. Incluye tres hechos del contexto, dos riesgos, la cadencia de entrega y cómo cambiarías el plan si el centro pidiera una auditoría formal.

Personas y responsabilidades en un equipo

El software no lo construye una sola persona ni una sola disciplina. Los títulos cambian entre organizaciones y una persona puede asumir varios papeles a la vez, así que importa comprender la **responsabilidad** de cada perfil, no memorizar un organigrama único.

PERFIL	RESPONSABILIDAD PRINCIPAL	COLABORA ESPECIALMENTE CON
Product manager / Product Owner	Maximizar el valor del producto y ordenar las necesidades	Usuarios, negocio y equipo
Jefatura de proyecto	Alcance, coordinación, recursos y riesgos	Interesados y equipo
Saltar al contenido principal	Descubrir, modelar y validar requisitos	Usuarios, diseño y calidad

PERFIL	RESPONSABILIDAD PRINCIPAL	COLABORA ESPECIALMENTE CON
Arquitecto o arquitecta	Decisiones estructurales y atributos de calidad	Desarrollo, operaciones y seguridad
UX/UI	Investigación, interacción, interfaz y accesibilidad	Usuarios y desarrollo de interfaz
Desarrollo	Diseñar, implementar, revisar y probar	Todo el equipo
QA / pruebas	Estrategia de calidad y evidencia de las pruebas	Análisis y desarrollo
DevOps / plataforma / SRE	Automatización, entrega, operación y fiabilidad	Desarrollo y operaciones
Seguridad	Amenazas, controles, revisión y respuesta a incidentes	Todas las fases
Scrum Master	Ayudar a que Scrum se comprenda y sea eficaz	Scrum Team y organización

Full-stack describe amplitud técnica entre cliente, servidor y datos; no implica dominarlo todo ni sustituye a las especialidades. *DevOps* es una cultura y un conjunto de prácticas para compartir la responsabilidad y acortar la retroalimentación; no es únicamente una persona que mantiene servidores.

Competencias transversales y matriz RACI

Además de las competencias técnicas, todo perfil necesita comunicación escrita y oral, trabajo colaborativo y revisión respetuosa, pensamiento crítico, capacidad de leer documentación técnica en inglés, responsabilidad sobre la privacidad, la accesibilidad y los sesgos, y aprendizaje continuo sin perseguir cada novedad.

Para dejar claro quién hace qué se usa una matriz **RACI**: R (*Responsible*) hace el trabajo, A (*Accountable*) aprueba y responde del resultado, C (*Consulted*) es consultada e I (*Informed*) es informada. Suele haber una sola A por tarea. Este es un ejemplo ligero:

TAREA	R (HACE)	A (APRUEBA)	C (SE LE CONSULTA)	I (SE LE INFORMA)
Validar los requisitos	Analista	Product Owner	Usuarios y desarrollo	Equipo
Aprobar una versión	QA	Product Owner	Desarrollo y seguridad	Operaciones
Responder a un incidente	DevOps y desarrollo	Jefatura de proyecto	Seguridad y protección de datos	Usuarios afectados

> COMPRUEBA LO QUE HAS ENTENDIDO
>
> Completa la matriz RACI para «elegir dónde se guardan los datos» y «retirar los datos al cerrar el servicio». ¿Por qué no debería haber dos personas con la A?

> LA SEGURIDAD NO ES UNA FASE: ACOMPAÑA A TODAS
>
> - El marco SSDF del NIST (SP 800-218) recomienda incorporar prácticas de seguridad en cada etapa en lugar de dejarlas para el final. Ejemplos por fase:
> - **Análisis**: minimizar los datos personales que se recogen. **Diseño**: pensar qué amenazas hay. **Codificación**: validar las entradas. **Pruebas**: analizar las dependencias y buscar vulnerabilidades conocidas. **Despliegue**: guardar contraseñas y claves fuera del repositorio. **Mantenimiento**: aplicar parches y vigilar avisos.

Más allá de la técnica: accesibilidad, sostenibilidad e inteligencia artificial

La **accesibilidad** debe convertirse en requisitos verificables. Las pautas WCAG 2.2 organizan los criterios web en cuatro principios: contenido perceptible, interfaz operable, información comprensible y tecnología robusta. Aunque una aplicación empiece en consola, debe usar mensajes claros, no depender solo del color y permitir el uso de tecnologías de apoyo.

El software **sostenible** evita trabajo y consumo innecesarios durante todo su ciclo: una arquitectura proporcionada, medir antes de optimizar, consultas y transferencias eficientes, apagar los recursos ociosos, mantener compatibles los dispositivos durante más tiempo y retirar con responsabilidad los datos y la infraestructura.

Sobre la **inteligencia artificial generativa**: puede sugerir requisitos, código, pruebas o documentación, pero no asume la responsabilidad profesional. Un uso responsable implica formular bien el objetivo y las restricciones, revisar la exactitud, la seguridad, las licencias y los sesgos, ejecutar pruebas independientes, registrar su uso cuando la política del centro lo exija y ser capaz de explicar y mantener el resultado. Una salida plausible no es una evidencia.

Saltar al contenido principal

ESTADO DEL SECTOR (2025)

El 18.º informe *State of Agile* de Digital.ai (28 de octubre de 2025) recoge que el 84 % de los encuestados usa IA en sus prácticas ágiles (frente al 68 % anterior), que solo el 49 % tiene salvaguardas para ese uso y que el 74 % aplica enfoques híbridos, mezclados o propios. Ojo: la muestra es de casi 350 personas, sobre todo coaches y consultores de grandes empresas. Sirve para ver una tendencia, no para describir todo el sector.

ERROR FRECUENTE — ÁGIL NO ES «SIN PLANIFICAR» NI SCRUM ES LO MISMO QUE ÁGIL

Ágil es un conjunto de valores; Scrum, Kanban y XP son formas concretas de aplicarlos. Un equipo ágil planifica con frecuencia y en poco alcance. Y Kanban no carece de reglas: los límites de trabajo en curso son su regla central.

ACTIVIDAD A13 · DINÁMICA KANBAN CON Y SIN LÍMITE (S7 · EQUIPOS DE 4)

Cada equipo tiene ocho tarjetas de historia (dibujar los elementos de una pantalla de móvil) que pasan por tres estaciones: **Analizar** (escribir el criterio de aceptación), **Dibujar** y **Probar** (comprobar el dibujo con el criterio). Ronda 1 de 8 minutos **sin límite**. Ronda 2 de 8 minutos con un máximo de **2 tarjetas en curso** en todo el tablero. Anota el tiempo hasta la primera tarjeta terminada y cuántas hay terminadas al final. Después discute qué cambió y dónde estaba el cuello de botella.

ACTIVIDAD A14 · RETROSPECTIVA (S7 · 10 MIN)

Tres columnas: **qué funcionó**, **qué no funcionó** y **qué probaremos**. Cada persona escribe una tarjeta por columna y el equipo elige una acción de mejora concreta para la unidad siguiente.

ACTIVIDAD A16 · TRIBUNAL DE METODOLOGÍAS (OPCIONAL · 10 MIN · EQUIPOS)

Cada equipo defiende **Cascada o V, Scrum o Kanban** para uno de estos casos: software de control de un dispositivo médico regulado; una web para validar una idea de negocio incierta; mantenimiento correctivo de 20 aplicaciones internas; un proyecto de aula con alumnado disponible para probar cada semana. La defensa incluye **por qué encaja**, **dos riesgos**, **cómo obtiene retroalimentación** y **qué condición haría cambiar de enfoque**. Después el tribunal puede proponer un modelo híbrido.

ACTIVIDAD A18 · INCIDENTE CON DATOS PERSONALES (OPCIONAL · FORO DE MOODLE)

Un prototipo ha publicado por error un fichero con datos personales. Representa en papel una reunión de 10 minutos con producto, desarrollo, seguridad y operaciones, calidad y la persona responsable de protección de datos. Entrega: **acciones inmediatas**, responsables y personas informadas, **evidencias** que conservar, **cambio preventivo** en el proceso o en las herramientas, y un mensaje **transparente y sin especular**.

Glosario de la unidad

TÉRMINO	DEFINICIÓN BREVE
Artefacto	Cualquier resultado que produce una fase: requisitos, código, informe de pruebas, paquete versionado
Bytecode	Código intermedio que ejecuta una máquina virtual, no la CPU directamente
Código objeto	Traducción a instrucciones máquina de un fichero fuente, con símbolos aún sin resolver
Enlazador	Combina ficheros objeto y bibliotecas y resuelve los símbolos para crear el ejecutable
JIT	Compilador que traduce a código nativo, durante la ejecución, lo que se usa con más frecuencia
Proceso	Programa cargado en memoria y en ejecución, con su espacio de direcciones
Requisito verificable	Requisito escrito de forma que se puede comprobar con una prueba y un dato
Trazabilidad	Enlace entre necesidad, requisito, diseño, prueba y evidencia
Validación	Comprobar que el producto resuelve la necesidad correcta
Verificación	Comprobar que el producto se ha construido de acuerdo con su especificación
WIP	Trabajo en curso: número de tarjetas empezadas y no terminadas
Tubería de CI	Secuencia automática de construcción, pruebas y publicación que se lanza con cada cambio

Saltar al contenido principal

FUENTES CONSULTADAS (SEPTIEMBRE DE 2026)

- BOE-A-2023-13221 — Real Decreto 405/2023: RA1 y criterios de evaluación del módulo.
- Scrum Guides — Revisiones: la edición de 2020 es la última listada.
- Digital.ai — 18th State of Agile Report (octubre de 2025).
- Stack Overflow Developer Survey 2025 — IA.
- Python 3.14 — Novedades: estado del JIT experimental y del modo sin GIL.
- NIST SP 800-218 — Secure Software Development Framework (SSDF): prácticas de seguridad en todo el ciclo.
- ISO/IEC/IEEE 12207:2026, segunda edición (29-04-2026), que sustituye a la de 2017; ISO/IEC 14764 para los tipos de mantenimiento.
- Política de soporte de .NET: .NET 10 LTS hasta el 14-11-2028.
- Ubuntu 26.04 LTS — Notas de versión: versiones de GCC, Python y OpenJDK.
- W3C — Web Content Accessibility Guidelines (WCAG) 2.2: los cuatro principios de accesibilidad.
- ISO/IEC 25010:2023 — Modelo de calidad del producto de sistemas y software (familia ISO/IEC 25000).
- Pro Git, edición en español: referencia de Git.
- Manifiesto por el Desarrollo Ágil de Software y sus doce principios.

Material complementario y licencia. Esta unidad incorpora contenidos adaptados de José Luis González Sánchez, *Entornos de Desarrollo – 01 – Desarrollo de Software*, con licencia Creative Commons BY-NC-SA 4.0. Esta adaptación se comparte bajo la misma licencia.

Los comandos y salidas de los bloques 1.1 a 1.6 se ejecutaron en Ubuntu 24.04 con GCC 13.3, OpenJDK 21 y Python 3.11.

Saltar al contenido principal

Ejercicios

Práctica gradual

24 ejercicios agrupados por bloque; dentro de cada bloque la dificultad crece, y el último integra toda la unidad. Intenta resolverlos antes de ver la solución.

CE1.a CE1.b CE1.c CE1.d CE1.e CE1.f CE1.g

01 Programa o proceso

Muy fácil CE1.a · Bloque 1.1

ENUNCIADO

Indica para cada afirmación si describe un **programa**, un **proceso** o ambos:

- Es un fichero que ocupa espacio en disco, como `/usr/bin/ls`.
- Tiene un identificador PID asignado por el núcleo.
- Puede existir aunque nadie lo haya ejecutado nunca.
- Dos usuarios que lanzan `ls` a la vez tienen cada uno el suyo, con memoria independiente.
- Dispone de pila, montón y bibliotecas mapeadas en un espacio de direcciones virtual.

AFIRMACIÓN	RESPUESTA	RAZÓN
Fichero en disco	Programa	Es el formato ELF almacenado, sin ejecutarse.
Tiene un PID	Proceso	El PID lo asigna el núcleo al crear el proceso.
Existe sin haberse ejecutado	Programa	Un proceso solo existe mientras se ejecuta.
Dos ejecuciones simultáneas de `ls`	Proceso	Son dos procesos del mismo programa, con espacios de direcciones separados.
Pila, montón y bibliotecas mapeadas	Proceso	Son partes del espacio de direcciones del proceso en ejecución.

> PUNTOS CLAVE
>
> Un programa es un fichero pasivo. Un proceso es ese programa cargado por el núcleo, con PID y espacio de direcciones propio. De un mismo programa pueden existir muchos procesos a la vez.

Saltar al contenido principal

02 ¿Dónde vive cada cosa?

Fácil CE1.a · Bloque 1.1

ENUNCIADO

Indica en qué región de la memoria del proceso (`.text` , `.rodata` , `.data` , `.bss` , pila o montón) se guarda cada elemento del programa siguiente. Después comprueba tu respuesta con `gcc -c regiones.c && nm regiones.o && size regiones.o` .

- `total` , `acumulado` y `llamadas`
- El texto `"hola"` y la variable `saludo`
- La variable `local` y la variable `datos`
- El bloque de 400 bytes que reserva `malloc`

regiones.c C

```c
#include <stdlib.h>

int total = 10;
int acumulado;
static int llamadas = 0;
const char *saludo = "hola";

int main(void) {
    int local = 5;
    int *datos = malloc(100 * sizeof(int));
    free(datos);
    return local + llamadas + acumulado + total + saludo[0];
}
```

ELEMENTO	REGIÓN	MOTIVO
`total`	`.data`	Global inicializada con un valor distinto de cero. `nm` la marca con D.
`acumulado`	`.bss`	Global sin inicializar: ocupa espacio solo en memoria. `nm` la marca con B.
`llamadas`	`.bss`	Estática inicializada a cero: el compilador la coloca en .bss. `nm` la marca con b minúscula (símbolo local).
`"hola"`	`.rodata`	Literal de cadena: datos de solo lectura. Aparece con `objdump -s -j .rodata` .
`saludo`	`.data`	El puntero es una global inicializada (apunta a la cadena). Ocupa 8 bytes.
`local` y `datos`	Pila	Variables locales: viven en el marco de `main` . `datos` es un puntero de 8 bytes.
Bloque de `malloc`	Montón	Memoria dinámica. No aparece en `nm` ni en `size` : se reserva al ejecutar.

Terminal BASH

```
$ gcc -c regiones.c && nm regiones.o
0000000000000000 B acumulado
                 U free
0000000000000004 b llamadas
0000000000000000 T main
                 U malloc
0000000000000000 D saludo
0000000000000000 D total
$ size regiones.o
   text    data     bss     dec     hex filename
    182      12       8     202      ca regiones.o
```

> PUNTOS CLAVE
>
> `size` cuenta .data (12 bytes: `total` y `saludo`) y .bss (8 bytes: `acumulado` y `llamadas`). La pila y el montón no aparecen porque no existen hasta que el proceso se ejecuta.

Saltar al contenido principal

03 Anatomía de una ejecución

Fácil CE1.a · Bloque 1.1

ENUNCIADO

Describe en unas 150 palabras, con un esquema de cajas y flechas, el recorrido de `java Hola` desde que pulsas intro hasta que aparece el texto en pantalla. Tu esquema debe incluir estos elementos y decir qué hace cada uno:

- El fichero ejecutable `java` y el fichero `Hola.class` en el almacenamiento.
- El sistema operativo (núcleo) y el proceso que crea.
- La memoria RAM y la CPU.
- La máquina virtual de Java, su intérprete y su compilador JIT.
- La biblioteca base de Java y la llamada al sistema que escribe en pantalla.
- El periférico de salida (terminal o pantalla).

1. El shell pide al núcleo que cree un proceso a partir del fichero ejecutable `java`, que está en el almacenamiento.
2. El núcleo carga en RAM las regiones del programa (código, datos), le da un PID y un espacio de direcciones propio y la CPU empieza a ejecutar sus instrucciones.
3. Ya como proceso, la JVM localiza `Hola.class` en el disco, lo carga en memoria y comprueba el bytecode.
4. Su intérprete ejecuta el bytecode. Si un método se repite mucho, el compilador JIT lo traduce a código nativo, que la CPU ejecuta directamente.
5. `System.out.println` pertenece a la biblioteca base de Java, que acaba llamando a `write()`, una llamada al sistema.
6. El núcleo comprueba permisos y entrega los bytes al controlador del terminal, que los muestra en pantalla (periférico de salida).

> PUNTOS CLAVE
>
> Ningún programa toca el hardware directamente: siempre pasa por el núcleo. Además del proceso de la JVM hay un segundo nivel, el del bytecode, que la máquina virtual ejecuta por su cuenta.

04 Fases y artefactos

Muy fácil CE1.b · Bloque 1.2

ENUNCIADO

Ordena las seis fases del desarrollo y asocia a cada una el artefacto que produce.

- Fases: pruebas, mantenimiento, diseño, despliegue, análisis de requisitos, codificación.
- Artefactos: informe de pruebas, historias de usuario, paquete instalable, diagrama de arquitectura, código y su historial, ticket de incidencia.

ORDEN	FASE	ARTEFACTO TÍPICO
1	Análisis de requisitos	Historias de usuario y especificación de requisitos
2	Diseño	Diagrama de arquitectura, contratos de API
3	Codificación	Código fuente y su historial en el control de versiones
4	Pruebas	Informe de pruebas
5	Despliegue	Paquete instalable o versión publicada
6	Mantenimiento	Tickets de incidencias y nuevas versiones

> PUNTOS CLAVE
>
> En metodologías iterativas y ágiles las fases se repiten muchas veces, pero cada historia de usuario sigue atravesándolas. Los artefactos son la evidencia de que una fase se ha realizado.

Saltar al contenido principal

05 Clasifica el mantenimiento

Fácil CE1.b · Bloque 1.2

ENUNCIADO

Clasifica cada situación como mantenimiento correctivo, evolutivo, adaptativo o perfectivo:

- La aplicación se cierra al introducir un divisor igual a cero.
- Se publica una versión de Ubuntu que cambia la ruta de una biblioteca y hay que ajustar la aplicación.
- El cliente pide poder exportar el listado de calificaciones a PDF.
- Se reescribe una consulta para que la pantalla cargue en 1 segundo en lugar de 3, sin cambiar lo que hace.

SITUACIÓN	TIPO	POR QUÉ
Se cierra al dividir entre cero	`Correctivo`	Se repara un defecto.
Cambio de ruta en el sistema operativo	`Adaptativo`	Se adapta la aplicación a un cambio del entorno.
Exportar a PDF	`Evolutivo`	Se añade una funcionalidad nueva.
Consulta más rápida	`Perfectivo`	Mejora una cualidad (rendimiento) sin cambiar la funcionalidad.

> PUNTOS CLAVE
>
> Solo el mantenimiento correctivo repara errores. Los otros tres son mejoras planificadas, y por eso el mantenimiento es una parte grande del coste total de un programa.

06 Requisitos que se pueden comprobar

Medio CE1.b · Bloque 1.2

ENUNCIADO

Para cada frase, decide si es un requisito **verificable**. Si no lo es, reescríbela para que lo sea e indica qué prueba lo comprobaría:

- «La aplicación será intuitiva.»
- «El listado responderá en menos de 2 s con 10 000 tareas.»
- «Se usará una base de datos moderna.»
- «El sistema permitirá marcar una tarea como completada.»
- «La aplicación será segura.»

FRASE	¿VERIFICABLE?	REESCRITURA Y PRUEBA
Intuitiva	`No`	«Una persona nueva completa el alta en menos de 3 minutos sin ayuda». Prueba de usabilidad con cinco personas.
Menos de 2 s con 10 000 tareas	`Sí (no funcional)`	Ya tiene magnitud y condición. Prueba de carga con 10 000 tareas en el equipo de referencia.
Base de datos moderna	`No`	Es una decisión de diseño y «moderna» no se mide. Como requisito: «los datos persisten tras cerrar y abrir la aplicación». Prueba: crear, cerrar, abrir y comparar.
Marcar una tarea como completada	`Sí (funcional)`	Prueba de aceptación: marcar una tarea y comprobar que aparece como completada.
Segura	`No`	«Las contraseñas se guardan cifradas con hash y tras 5 intentos fallidos la cuenta se bloquea 15 minutos». Prueba automática de los dos comportamientos.

> PUNTOS CLAVE
>
> Un requisito verificable tiene un sujeto, una condición y una magnitud o comportamiento observable. Los adjetivos («intuitivo», «moderno», «seguro») no se pueden probar.

Saltar al contenido principal

07 Verificar, validar y mantener

Medio CE1.b · Bloque 1.2

ENUNCIADO

Clasifica cada situación como **verificación** o **validación**. Después nombra el tipo de mantenimiento (correctivo, evolutivo, adaptativo, perfectivo o preventivo) de las dos últimas:

- Comprobar que el código cumple el documento de diseño mediante una revisión.
- Enseñar el prototipo al cliente y preguntarle si resuelve su problema.
- Ejecutar las pruebas unitarias escritas a partir de la especificación.
- Un grupo de usuarios reales prueba la versión beta y confirma que les sirve.
- Sustituir una biblioteca que dejará de tener soporte, aunque hoy funciona bien.
- Cambiar la ruta de guardado porque la nueva versión del sistema operativo la ha movido.

SITUACIÓN	CLASIFICACIÓN	RAZÓN
Revisión frente al diseño	Verificación	Compara el producto con su especificación: ¿está bien construido?
Prototipo al cliente	Validación	Comprueba la necesidad real: ¿es el producto correcto?
Pruebas unitarias desde la especificación	Verificación	Contrastan el código con lo especificado.
Beta con usuarios reales	Validación	Los usuarios confirman que resuelve su necesidad.
Sustituir la biblioteca sin soporte	Mantenimiento preventivo	Evita fallos futuros antes de que ocurran.
Nueva ruta por el sistema operativo	Mantenimiento adaptativo	Responde a un cambio del entorno sin cambiar lo que hace la aplicación.

> PUNTOS CLAVE
>
> Verificar mira hacia la especificación; validar mira hacia la necesidad. Un producto puede estar perfectamente verificado y no servir a nadie.

Saltar al contenido principal

08 Fuente, objeto o ejecutable

Fácil CE1.c · Bloque 1.3

ENUNCIADO

Completa la tabla para cada fichero de la cadena de compilación de `saludo.c` : tipo de fichero, si es texto o binario y si se puede lanzar.

- `saludo.c` , `saludo.i` , `saludo.s` , `saludo.o` , `saludo` y `Saludo.class` (versión Java del mismo programa).

FICHERO	TIPO	TEXTO O BINARIO	¿SE PUEDE LANZAR?
`saludo.c`	Código fuente en C	Texto	No
`saludo.i`	Fuente preprocesado (`#include` expandido)	Texto	No
`saludo.s`	Código ensamblador	Texto	No
`saludo.o`	Código objeto reubicable, incompleto (`U printf`)	Binario	No: falta enlazar
`saludo`	Ejecutable completo (ELF)	Binario	Sí, el cargador resuelve las bibliotecas
`Saludo.class`	Bytecode para la JVM	Binario	Sí, pero con `java Saludo` , no directamente

> PUNTOS CLAVE
>
> Objeto no significa orientado a objetos: es el resultado de compilar un fichero. El bytecode es ejecutable, pero por una máquina virtual y no por la CPU.

09 Reconstruye la cadena

Medio CE1.c · Bloque 1.3

ENUNCIADO

Escribe los cuatro comandos que producen, en orden, `saludo.i` , `saludo.s` , `saludo.o` y el ejecutable `saludo` a partir de `saludo.c` , cada uno partiendo del fichero anterior. Después responde:

- ¿En qué fichero aparece por primera vez `U printf` al ejecutar `nm` ?
- ¿Qué fichero es más grande, `saludo.c` o `saludo.i` ? ¿Por qué?

Terminal BASH

```
$ gcc -E saludo.c -o saludo.i      # preprocesado
$ gcc -S saludo.i -o saludo.s      # compilación a ensamblador
$ gcc -c saludo.s -o saludo.o      # ensamblado a código objeto
$ gcc saludo.o -o saludo           # enlazado
$ nm saludo.o | grep printf
                 U printf
$ wc -c saludo.c saludo.i
   121 saludo.c
 21334 saludo.i
```

> RESPUESTAS
>
> - `U printf` aparece por primera vez en `saludo.o` , porque el código objeto contiene la llamada pero no la función.
> - `saludo.i` es mucho mayor (21 334 bytes frente a 121) porque el preprocesador expande `#include <stdio.h>` con todas sus declaraciones.

> PUNTOS CLAVE
>
> Cada paso hace una cosa: expandir, traducir a ensamblador, traducir a binario y unir con las bibliotecas. `gcc saludo.c -o saludo` encadena los cuatro sin dejar los ficheros intermedios.

Saltar al contenido principal

10 El símbolo que no se resuelve

Difícil CE1.c · Bloque 1.3

ENUNCIADO

Con el programa `doble.c`, ejecuta `gcc -c doble.c` y después `gcc doble.o -o doble`. Explica el resultado y responde:

- ¿Por qué el primer comando termina bien y el segundo no?
- ¿Qué muestra `nm doble.o` para `contador`, `doble`, `oculto` y `sqrt`?
- ¿Cómo se corrige el error de enlazado?

doble.c C

```c
#include <stdio.h>
#include <math.h>

int contador = 5;
static int oculto = 1;

int doble(int x) {
    return 2 * x;
}

int main(void) {
    printf("%d %f\n", doble(contador), sqrt((double)contador));
    return oculto;
}
```

Terminal BASH

```
$ gcc -c doble.c
$ gcc doble.o -o doble
/usr/bin/ld: doble.o: in function `main':
doble.c:(.text+0x38): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
$ nm doble.o
0000000000000000 D contador
0000000000000000 T doble
0000000000000012 T main
0000000000000004 d oculto
                 U printf
                 U sqrt
$ gcc doble.o -o doble -lm
$ ./doble ; echo $?
10 2.236068
1
```

RESPUESTAS

- Compilar solo necesita que el compilador conozca la declaración de `sqrt` (viene de `math.h`). Que la función exista se comprueba al enlazar.
- `contador` es D (dato inicializado global), `doble` y `main` son T (código), `oculto` es d minúscula (dato local del fichero por ser `static`), y `printf` y `sqrt` son U (indefinidos).
- Se corrige añadiendo la biblioteca matemática al enlazar con `-lm`. `printf` no da error porque libc se enlaza siempre por defecto.

PUNTOS CLAVE

Un fichero objeto puede tener símbolos indefinidos. El enlazador es quien exige que todos se resuelvan. La letra de `nm` (mayúscula o minúscula) indica si el símbolo es global o local al fichero.

Saltar al contenido principal

11 ¿Qué tipo de error es?

Fácil CE1.c · Bloque 1.3

ENUNCIADO

Clasifica cada mensaje como error **léxico**, **sintáctico**, **semántico** (de tipos), de **enlazado** o de **ejecución**, y di qué herramienta lo detecta:

- `error: stray '@' in program` (gcc)
- `error: expected ',' or ';' before 'return'` (gcc)
- `error: incompatible types: String cannot be converted to int` (javac)
- `undefined reference to` sqrt`` (enlazador)
- `Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException` (java)

MENSAJE	TIPO	QUIÉN LO DETECTA
`stray '@'`	Léxico	El analizador léxico del compilador: un carácter que no pertenece al lenguaje.
`expected ',' or ';'`	Sintáctico	El analizador sintáctico: la estructura de la sentencia no es válida.
`incompatible types`	Semántico	El análisis semántico del compilador: los tipos no encajan.
`undefined reference`	Enlazado	El enlazador: falta resolver un símbolo entre ficheros objeto y bibliotecas.
`ArrayIndexOutOfBoundsException`	Ejecución	La JVM, con el programa ya en marcha.

PUNTOS CLAVE

El mensaje dice en qué momento falla la cadena. Cuanto antes se detecta un error, más barato es corregirlo.

12 Lee el número mágico

Fácil CE1.d · Bloque 1.4

ENUNCIADO

Al ejecutar `od -An -tx1 -N8 Hola.class` obtienes `ca fe ba be 00 00 00 41` . Responde:

- ¿Cuál es la versión mayor del fichero y con qué versión de Java se compiló?
- ¿Qué valor esperarías en los dos últimos bytes si se hubiera compilado con Java 17? ¿Y con Java 8?

RESPUESTAS

- Los cuatro primeros bytes (`ca fe ba be`) son el número mágico de los ficheros .class. Los dos siguientes son la versión menor (0) y los dos últimos la versión mayor: 0x41 = 65. Como versión mayor = versión de Java + 44, se compiló con Java 21.
- Con Java 17 la versión mayor sería 61 (0x3d): `00 00 00 3d` . Con Java 8 sería 52 (0x34): `00 00 00 34` .

Terminal BASH

```
$ javac --release 17 Hola.java
$ od -An -tx1 -N8 Hola.class
 ca fe ba be 00 00 00 3d
```

PUNTOS CLAVE

Una JVM antigua no puede cargar una clase con una versión mayor que la que conoce. Por eso se puede compilar con `--release` para una versión anterior y mantener la compatibilidad.

Saltar al contenido principal

13 Lee el bytecode

Medio CE1.d · Bloque 1.4

ENUNCIADO

`javap -c Doble` muestra el bytecode del método `public static int doble(int x) { return x * 2; }`. Explica qué hace cada instrucción y escribe el contenido de la pila cuando se ejecuta con `x = 21`. ¿Qué instrucción cambiaría en la primera línea si el método no fuera `static`?

Terminal · javap -c Doble BASH

```
public static int doble(int);
  Code:
     0: iload_0
     1: iconst_2
     2: imul
     3: ireturn
```

INSTRUCCIÓN	QUÉ HACE	PILA DESPUÉS
`iload_0`	Carga en la pila la variable local 0, el parámetro `x`	[21]
`iconst_2`	Carga la constante entera 2	[21, 2]
`imul`	Saca los dos valores, los multiplica y apila el resultado	[42]
`ireturn`	Devuelve el entero que está en la cima de la pila	vacía

MÉTODO DE INSTANCIA

Si el método no fuera `static`, la variable local 0 sería `this` y `x` pasaría a ser la variable 1: la primera instrucción sería `iload_1`.

PUNTOS CLAVE

La JVM es una máquina de pila: cargar, operar y devolver. Los nombres exactos de las instrucciones no hay que memorizarlos, pero sí la idea de cómo se evalúa una expresión.

Saltar al contenido principal

14 Tres cosas «virtuales»

Fácil CE1.d · Bloque 1.4

ENUNCIADO

Indica si cada afirmación corresponde a una máquina virtual de **proceso** (JVM, CLR), de **sistema** (VirtualBox) o a un **contenedor**, y razona:

- Ejecuta el mismo `Hola.class` en Windows y en Linux sin recompilar.
- Instala un Ubuntu completo dentro de otro sistema operativo, con su propio núcleo.
- Comparte el núcleo del anfitrión y arranca en menos de un segundo.
- Un contenedor puede ejecutar un núcleo distinto del anfitrión. ¿Verdadero o falso?

AFIRMACIÓN	RESPUESTA	RAZÓN
Mismo `.class` en Windows y Linux	`Proceso`	El bytecode es portable; la JVM de cada sistema lo ejecuta.
Ubuntu completo con su núcleo	`Sistema`	Simula un ordenador entero para instalar un sistema operativo.
Comparte núcleo y arranca en menos de 1 s	`Contenedor`	Es un proceso aislado del anfitrión, no una máquina simulada.
Contenedor con otro núcleo	`Falso`	Un contenedor usa el núcleo del anfitrión. Si necesitas otro, necesitas una máquina virtual.

PUNTOS CLAVE

Comparten nombre, no objetivo. La de proceso ejecuta un programa con código intermedio, la de sistema simula hardware y el contenedor aísla procesos en el mismo núcleo.

15 Etiqueta los lenguajes

Fácil CE1.e · Bloque 1.5

ENUNCIADO

Clasifica **Go**, **PHP** y **Haskell** según cuatro ejes: nivel de abstracción, estrategia de ejecución, paradigma y sistema de tipos. Marca con un asterisco las etiquetas que consideres discutibles.

- Usa la documentación oficial de cada lenguaje como evidencia y cita la fuente.

LENGUAJE	NIVEL	EJECUCIÓN	PARADIGMA	TIPOS
Go	Alto	Compilado a código nativo	Imperativo y concurrente (goroutines)	Estático
PHP	Alto	El motor compila a instrucciones internas y las ejecuta en su máquina virtual*	Multiparadigma: imperativo y orientado a objetos	Dinámico (con declaraciones de tipo opcionales)*
Haskell	Alto	Compilado a nativo con GHC (también existe intérprete, GHCi)*	Funcional puro	Estático, con inferencia de tipos

PUNTOS CLAVE

Una clasificación honesta usa varios ejes y admite matices. Que un lenguaje tenga un intérprete no impide que tenga también un compilador nativo.

Saltar al contenido principal

16 Desmonta tres mitos

Medio CE1.e · Bloque 1.5

ENUNCIADO

Refuta cada afirmación con una evidencia que puedas comprobar en el laboratorio:

- «Python es interpretado y no genera código intermedio.»
- «Java se compila directamente a código máquina.»
- «Un lenguaje es compilado o interpretado, pero nunca las dos cosas.»

MITO	POR QUÉ ES FALSO	EVIDENCIA
Python no genera código intermedio	CPython compila a bytecode y lo ejecuta en su máquina virtual	`python3 -m py_compile suma.py` crea un .pyc en `__pycache__` ; `dis` muestra sus instrucciones
Java compila a código máquina	`javac` genera bytecode, que ejecuta la JVM (con JIT)	`od -An -tx1 -N8 Suma.class` muestra `ca fe ba be` ; `javap -c Suma` muestra `iload_0` , `iadd` …
Compilado o interpretado, nunca ambos	Es una propiedad de la implementación, no del lenguaje	Existen intérpretes de C y compiladores nativos de Java (GraalVM Native Image); HotSpot interpreta y compila a la vez

PUNTOS CLAVE

Cuando alguien dice que un lenguaje es compilado o interpretado, en realidad habla de la implementación más habitual. Justifica siempre las etiquetas con una evidencia.

17 ¿Dinámico o solo inferido?

Fácil CE1.e · Bloque 1.5

ENUNCIADO

Indica si cada fragmento compila o se ejecuta sin error y si el lenguaje comprueba los tipos antes de ejecutar. ¿Hay tipado dinámico o solo inferencia de tipos?

Fragmentos JAVA Y PYTHON

```
// Java (JDK 21)
var n = 3;
n = "hola";

# Python 3
n = 3
n = "hola"
```

FRAGMENTO	RESULTADO	RAZÓN
Java con `var`	No compila	`var` solo deduce el tipo (`int`); después es fijo. El error es `incompatible types: String cannot be converted to int` , detectado antes de ejecutar.
Python	Se ejecuta	El tipo está en el valor, no en la variable: `n` puede apuntar a un `int` y luego a un `str` . La comprobación es dinámica.

PUNTOS CLAVE

Inferir el tipo no es tener tipado dinámico. Con inferencia, el compilador deduce el tipo y lo mantiene; con tipado dinámico el tipo cambia con el valor.

Saltar al contenido principal

18 La herramienta adecuada

Muy fácil CE1.f · Bloque 1.6

ENUNCIADO

Asocia cada tarea con la familia de herramientas que la resuelve: depurador, gestor de dependencias, control de versiones, herramienta de construcción, IDE (editor con servidor de lenguaje) y tablero de tareas.

- Parar la ejecución en una línea y ver el valor de las variables.
- Instalar una biblioteca y las que ella necesita.
- Guardar el historial de cambios y colaborar en el mismo código.
- Automatizar los pasos de compilación según qué ficheros han cambiado.
- Autocompletar y marcar errores mientras se escribe.
- Planificar y seguir el estado de las tareas del equipo.

TAREA	FAMILIA	EJEMPLOS
Parar y ver variables	Depurador	gdb, el depurador del IDE
Instalar una biblioteca y sus dependencias	Gestor de dependencias	pip, npm, Maven
Historial de cambios y colaboración	Control de versiones	Git
Automatizar la compilación	Herramienta de construcción	make, Maven, Gradle
Autocompletar y marcar errores	IDE / editor con servidor de lenguaje (LSP)	VS Code, IntelliJ IDEA
Planificar y seguir tareas	Tablero de tareas	Tablero Kanban, GitHub Projects, Jira

PUNTOS CLAVE

Cada familia cubre una fase distinta del ciclo de desarrollo. Evaluar una herramienta es comprobar en qué fase interviene y qué límites tiene.

Saltar al contenido principal

19 El grafo de dependencias

Medio CE1.f · Bloque 1.6

ENUNCIADO

En un entorno virtual nuevo, instala `requests==2.32.3` y averigua con qué paquetes queda instalado. Responde:

- ¿Cuántos paquetes hay en el entorno y cuáles son?
- ¿Cuál es la dependencia directa y cuáles las transitivas?
- ¿Cómo sabes de cuáles depende `requests` ?

Terminal BASH

```
$ python3 -m venv .venv
$ .venv/bin/pip install requests==2.32.3
$ .venv/bin/pip list
Package            Version
------------------ ---------
certifi            (versión)
charset-normalizer (versión)
idna               (versión)
pip                (versión)
requests           2.32.3
urllib3            (versión)
$ .venv/bin/pip show requests | grep Requires
Requires: certifi, charset-normalizer, idna, urllib3
```

RESPUESTAS

- Además de `pip` , hay cinco paquetes: `requests` , `certifi` , `charset-normalizer` , `idna` y `urllib3` .
- La dependencia directa es `requests` , la que pediste. `certifi` , `charset-normalizer` , `idna` y `urllib3` son transitivas: las necesita `requests` .
- Con `pip show requests` , en la línea `Requires` .

PUNTOS CLAVE

Los números de versión de las dependencias transitivas dependen del día en que se instala. Un fichero `requirements.txt` con versiones fijas garantiza el mismo entorno en todos los equipos.

Saltar al contenido principal

20 Un commit por dentro

Difícil CE1.f · Bloque 1.6

ENUNCIADO

Crea un repositorio con `git init -b main`, añade un fichero `a.txt` con el texto `hola`, confirma el cambio y responde con lo que veas por pantalla:

- ¿Qué contiene un commit según `git cat-file -p HEAD`?
- ¿Qué contiene el fichero `.git/refs/heads/main` y qué relación tiene con el commit?
- ¿Qué aparece al inspeccionar el árbol con `git cat-file -p 'HEAD^{tree}'`?

Terminal BASH

```
$ git init -b main && echo hola > a.txt
$ git add a.txt && git commit -m "Primer commit"
$ git cat-file -t HEAD
commit
$ git cat-file -p HEAD
tree 40ce32801aa3d5001925dc4dc342210ec3b57b2e
author Alumno <alumno@ejemplo.es> 1789820731 +0200
committer Alumno <alumno@ejemplo.es> 1789820731 +0200

Primer commit
$ cat .git/refs/heads/main
2b1dd8e99555414610d1af5ce84d81b513a93756
$ git cat-file -p 'HEAD^{tree}'
100644 blob 5c1b14949828006ed75a3e8858957f86a2f7e2eb    a.txt
```

RESPUESTAS

- Un commit es un objeto con una referencia a un árbol (`tree`), el autor, el confirmador, la fecha y el mensaje. Los identificadores serán distintos en tu equipo.
- `refs/heads/main` contiene el identificador del último commit de la rama: una rama es un puntero a un commit.
- El árbol lista los ficheros del proyecto: un `blob` por fichero, con su permiso, su identificador y su nombre.

PUNTOS CLAVE

Git guarda instantáneas del proyecto (blobs y árboles) y los commits las enlazan. Una rama es solo un fichero de 40 caracteres, por eso crear ramas es tan barato.

Saltar al contenido principal

21 Biblioteca, framework, runtime o SDK

Fácil CE1.f · Bloque 1.6

ENUNCIADO

Clasifica cada elemento como **biblioteca**, **framework**, **runtime** o **SDK**. Después decide si cada situación es integración continua, entrega continua o despliegue continuo:

- `java.lang.Math`
- JUnit, que descubre y llama a tus métodos de prueba
- El JRE (la JVM con las bibliotecas base)
- El JDK (`javac` , `javap` , `jar` y el JRE)
- Cada cambio se integra y se prueba automáticamente, pero publicar exige pulsar un botón.
- Todo cambio que supera la tubería se publica sin intervención humana.

ELEMENTO O SITUACIÓN	RESPUESTA	RAZÓN
`java.lang.Math`	`Biblioteca`	Tu código llama a sus funciones.
JUnit	`Framework`	Controla el flujo: él llama a tu código (inversión de control).
JRE	`Runtime`	Solo permite ejecutar programas ya hechos.
JDK	`SDK`	Incluye compilador y herramientas para desarrollar.
Cambios probados; publicar con botón	`Entrega continua`	Siempre está listo para publicar, pero la publicación la decide una persona.
Publicación automática	`Despliegue continuo`	No hay intervención humana entre la tubería y producción.

PUNTOS CLAVE

La aplicación llama a una biblioteca; un framework llama a la aplicación. El SDK sirve para construir y el runtime para ejecutar.

22 ¿Ágil o secuencial?

Fácil CE1.g · Bloque 1.7

ENUNCIADO

Para cada proyecto, elige un enfoque (ágil o dirigido por plan, como cascada o en V) y justifícalo con dos características del escenario:

- Software de control de frenado de un tren, con certificación regulatoria y requisitos cerrados.
- Aplicación de reservas para un restaurante nuevo, cuyo dueño está disponible cada semana y todavía no sabe qué quiere.
- Migración de una base de datos con requisitos completamente definidos y sin cambios previstos.

PROYECTO	ENFOQUE	JUSTIFICACIÓN
Control de frenado	`Dirigido por plan (por ejemplo en V)`	Requisitos estables y certificación que exige documentación y trazabilidad previas.
Reservas del restaurante	`Ágil (Scrum o Kanban)`	Requisitos cambiantes y cliente disponible para dar feedback en cada incremento.
Migración de datos	`Dirigido por plan o iterativo sencillo`	Alcance cerrado y sin cambios previstos; se puede planificar de principio a fin.

PUNTOS CLAVE

Ágil no significa sin planificar ni es mejor en todos los casos. Se elige según la incertidumbre de los requisitos, la disponibilidad del cliente y las exigencias regulatorias.

Saltar al contenido principal

23 Incidente con datos personales

Medio CE1.g · Bloque 1.7

ENUNCIADO

Un prototipo ha publicado por error un fichero con datos personales del alumnado. Prepara la respuesta de una reunión de diez minutos entre producto, desarrollo, seguridad y operaciones, calidad y la persona responsable de protección de datos. Responde por escrito:

- Qué **acciones inmediatas** hay que tomar.
- Quién es responsable de cada una y a quién hay que informar.
- Qué **evidencias** hay que conservar.
- Qué **cambio preventivo** en el proceso o en las herramientas evitaría que se repita.
- Redacta un mensaje **transparente y sin especular** sobre lo ocurrido.

APARTADO	RESPUESTA ORIENTATIVA
Acciones inmediatas	Retirar el fichero y los enlaces, revocar accesos, comprobar si ha sido descargado y avisar a la persona responsable de protección de datos.
Responsables e informados	Operaciones retira el fichero; seguridad investiga el alcance; producto coordina la comunicación; el responsable de protección de datos decide las notificaciones necesarias.
Evidencias	Registros de acceso, línea temporal con horas, versión publicada y quién la publicó. No se borran registros para «limpiar».
Cambio preventivo	Revisión previa de lo que se publica, análisis automático en la tubería de CI de ficheros con datos personales, y minimizar los datos usados en pruebas.
Mensaje	Qué ocurrió, qué datos estuvieron expuestos y durante cuánto tiempo, qué se ha hecho ya y qué se va a hacer. Sin culpar a nadie ni especular sobre la causa hasta tener datos.

> PUNTOS CLAVE
>
> La respuesta a un incidente es un trabajo de roles: cada persona sabe qué hace. La transparencia se basa en hechos comprobados; especular empeora la situación.

Saltar al contenido principal

24 Informe técnico integrador

Muy difícil CE1.a-g · Bloque 1.1–1.7

ENUNCIADO

Elige uno de tus laboratorios y redacta un informe técnico de una página que responda a estas seis preguntas, con evidencias (comandos y salidas) de tu propio equipo:

- **Ejecución (CE1a).** ¿Qué región de memoria contiene el código y cuál las variables locales? Enséñalo con `/proc/PID/maps` o con `size`.
- **Fases (CE1b).** ¿En qué fase estás cuando compilas, cuando pruebas y cuando publicas? ¿Qué artefacto produce cada una?
- **Cadena (CE1c y CE1d).** Compara qué genera y qué ejecuta cada uno de C, Java y Python para un mismo `hola`. Mide el arranque de cada uno con `time`.
- **Lenguajes (CE1e).** Clasifica los tres lenguajes según cuatro ejes con una evidencia por etiqueta.
- **Herramientas (CE1f).** Elige una herramienta que hayas usado y evalúala con los seis criterios de la matriz del Lab 3.
- **Metodología (CE1g).** Plantea cómo organizarías estas tareas en un tablero Kanban con un límite de trabajo en curso y qué límite pondrías.

GUÍA DE CORRECCIÓN (MODELO DE RESPUESTA)

- **Ejecución:** el código está en la región `r-xp` del ejecutable y las variables locales en `[stack]`. `size` muestra text, data y bss del fichero, pero no la pila ni el montón.
- **Fases:** compilar es codificación, ejecutar pruebas es la fase de pruebas y publicar es despliegue. Artefactos: código, informe de pruebas y paquete.
- **Cadena:** C genera un ejecutable nativo que ejecuta la CPU; Java genera bytecode (.class) que ejecuta la JVM con JIT; Python genera bytecode (.pyc) que ejecuta CPython. Orden de magnitud del arranque de un `hola`: C unos milisegundos, Python decenas de milisegundos y Java alrededor de medio centenar; los valores exactos dependen del equipo y no se deben exigir.
- **Lenguajes:** cada etiqueta con una evidencia comprobable (`file`, `javap`, `dis`) o documentación oficial, con lo discutible marcado.
- **Herramientas:** seis criterios (fase, funcionalidad clave, evidencia, límites, alternativa y veredicto de 0 a 3 razonado).
- **Metodología:** columnas del tablero, límite de trabajo en curso (por ejemplo 2 o 3 tarjetas) y qué cuello de botella se espera ver.

Terminal BASH

```
$ time ./hola
$ time java Hola
$ time python3 hola.py
```

REFLEXIÓN FINAL

Este ejercicio integra los siete criterios de la unidad. Si eres capaz de explicar con tus propias salidas cómo un programa llega desde el código hasta la CPU, y con qué herramientas y metodología se construye, has alcanzado el RA1.